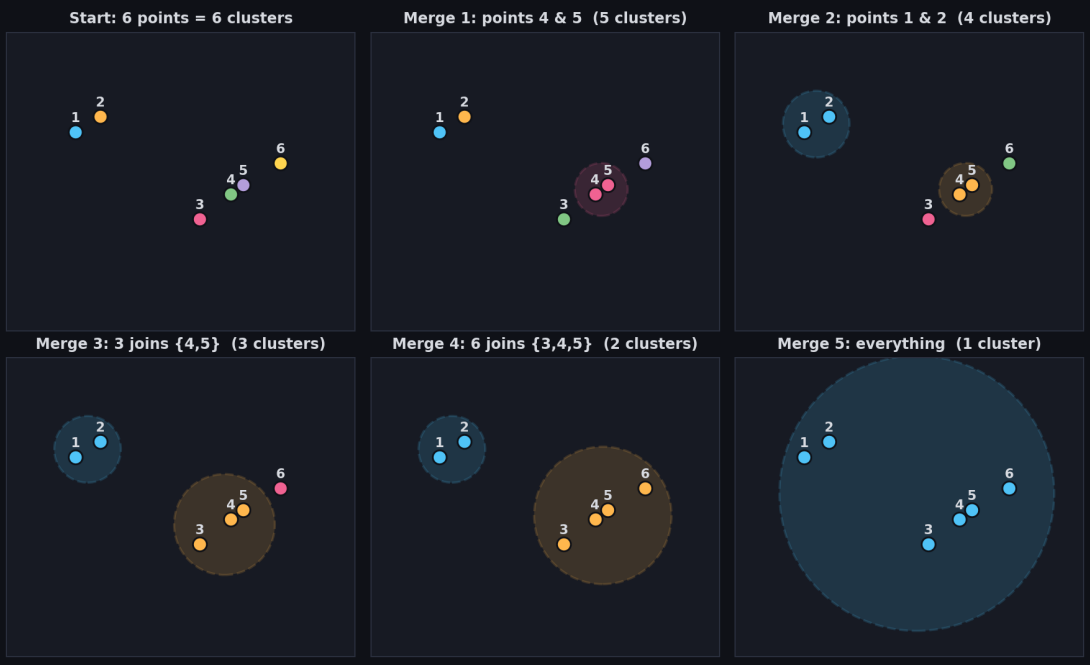
A COMPLETE VISUAL TUTORIAL · BASIC → PRO

Hierarchical Clustering
&
DBSCAN

Why K-Means is not enough · how dendrograms work · linkage methods · density-based clustering
core, border and noise points · math, visuals, worked examples and code



Built from two YouTube lecture scripts by Nitish (CampusX), reorganised into a step-by-step tutorial, with extra maths, figures, worked numbers and pro-level notes added. Nothing from the scripts was left out.

Contents

Contents 2

How to use this tutorial 4

Chapter 0 · Foundations 5

0.1 What is clustering? 5

0.2 Distance: the language of clustering 5

0.3 K-Means in two minutes 6

0.4 Where K-Means fails (the reason for everything that follows) 6

Chapter 1 · Hierarchical Clustering 10

1.1 The big idea (Level 1 · Basic) 10

1.2 Two flavours 10

1.3 Agglomerative clustering, step by step (6 points) 10

1.4 The dendrogram: your record of every merge 11

1.5 Divisive clustering (the opposite direction) 12

1.6 The exact algorithm (Level 2 · Intermediate) 13

1.7 Four ways to measure distance between clusters (linkage) 15

1.8 The four linkages on real shapes 17

1.9 How many clusters? Reading the dendrogram 18

1.10 scikit-learn: the AgglomerativeClustering class 19

1.11 Hands-on project: Mall customers 20

1.12 Strengths and limitations 21

1.13 Pro level (Level 3) 22

Chapter 2 · DBSCAN 25

2.1 Why do we need yet another algorithm? 25

2.2 Density-based clustering: the idea 25

2.3 The two knobs: ε (epsilon) and MinPts 26

2.4 Three kinds of points: core, border, noise 27

2.5 Density-connected points 29

2.6 The DBSCAN algorithm (4 steps) 30

2.7 DBSCAN in scikit-learn 31

2.8 Choosing ε and MinPts (Level 3 · Pro) 32

2.9 Advantages and limitations 34

2.10 A visualisation tool that makes it click 35

Chapter 3 · Putting It All Together 36

3.1 K-Means vs Hierarchical vs DBSCAN 36

3.2 Which one should I use? 38

3.3 Interview questions (with short answers) 38

3.4 Cheat sheet 39

3.5 Practice and mini-projects 39

3.6 Glossary and notation 40

How to use this tutorial

Your two video scripts were spoken explanations, so the order jumped around and the pictures were only described. Here everything is rebuilt as a **ladder**: each step needs only the steps below it. Read top to bottom the first time; later, jump to what you need.

Level	What you learn	Where
Level 0 · Foundations	What clustering is, distance, K-Means recap, and exactly where K-Means breaks	Chapter 0
Level 1 · Basic	Hierarchical clustering intuition: merging points, the dendrogram, agglomerative vs divisive	Ch. 1.1 – 1.5
Level 2 · Intermediate	The full algorithm, proximity matrix, 4 linkage methods, choosing k, scikit-learn, Mall-customer project	Ch. 1.6 – 1.12
Level 3 · Advanced / Pro	Lance–Williams formula, Ward maths, cophenetic correlation, complexity, scaling tricks	Ch. 1.13
Part 2 · DBSCAN	Density, ε and MinPts, core/border/noise, reachability, the 4-step algorithm, code, tuning, OPTICS/HDBSCAN	Chapter 2
Part 3 · Wrap-up	Side-by-side comparison, decision flowchart, interview Q&A, cheat sheet, exercises, glossary	Chapter 3

Boxes you will see

INTUITION
INTUITION boxes give the plain-English picture before any formula.

MATH
MATH boxes show the formula. Every symbol is explained right below it, so you can skip them at first and return later.

WATCH OUT
WATCH OUT boxes mark common mistakes and limits.

PRO TIP
PRO TIP boxes are Level-3 extras for interviews and real projects.

CORRECTION / UPDATE
CORRECTION / UPDATE boxes flag places where the spoken script was loose or where newer library versions changed something. They are marked clearly so nothing is silently altered.

NOTE
About the figures: every picture was generated with real code (scikit-learn, SciPy). The Mall-customer figures use a synthetic stand-in with the same structure (5 customer groups in income vs spending score), because the original CSV is not bundled here. The code listings work unchanged on the real file.

Chapter 0 · Foundations

Before hierarchical clustering makes sense, you need four small ideas: what clustering is, how to measure distance, how K-Means works, and where it fails. That last one is the whole reason the rest of this tutorial exists.

0.1 What is clustering?

Clustering means grouping data points so that points in the *same* group are similar and points in *different* groups are not. It is **unsupervised learning**: nobody gives you the answer labels. The algorithm must discover the groups itself.

> **EXAMPLE**
> A shop has 200 customers. For each one you know yearly income and a spending score. Nobody told you the customer types. Clustering may reveal: *careful low-income savers, high-income big spenders, average customers...* Marketing can then treat each group differently.

Two things every clustering method must decide: **(1)** how to measure *similarity* (almost always a distance), and **(2)** what a *cluster* is (a blob around a centre? a chain of nearby points? a crowded region?). Different answers to (2) give different algorithms. That is exactly the story of this tutorial.

0.2 Distance: the language of clustering

Small distance = similar. For two points x and y with n features, the three most common distances are:

$$d_{euclid}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$ Euclidean (L2)

$$d_{manhattan}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$ Manhattan (L1)

$$d_{cosine}(x, y) = 1 - \frac{x \cdot y}{\|x\| \, \|y\|}$$ Cosine

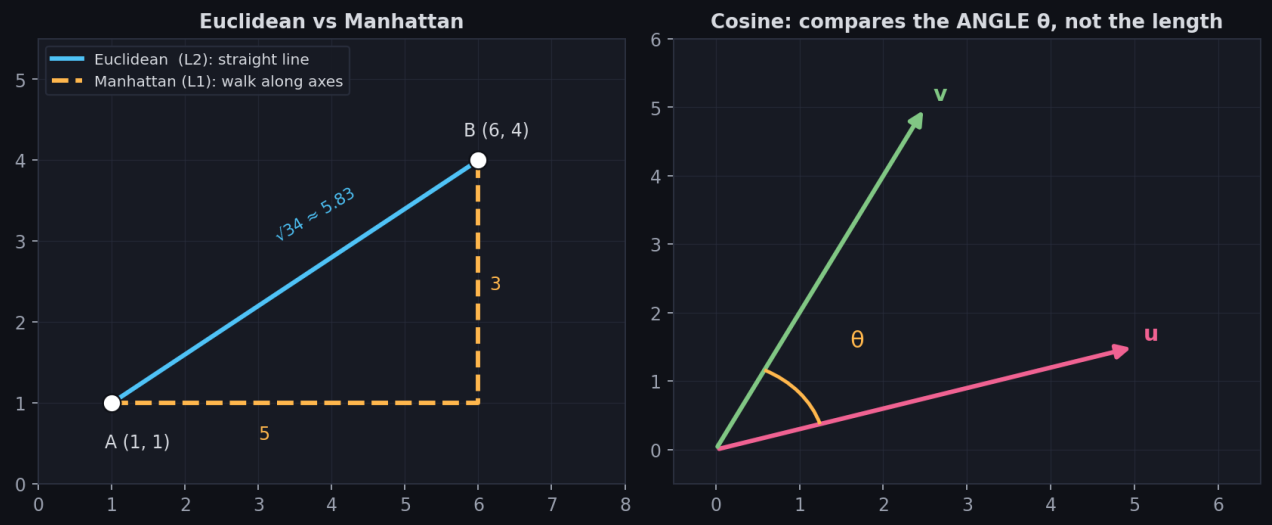


Figure 0.1 – Euclidean is the straight line, Manhattan walks along the axes (like city blocks), cosine only looks at the angle between the two vectors.

EXAMPLE

A = (1, 1) and B = (6, 4). Euclidean = $\sqrt{((6-1)^2 + (4-1)^2)} = \sqrt{(25+9)} = \sqrt{34} \approx 5.83$. Manhattan = $|6-1| + |4-1| = 5 + 3 = 8$. *Manhattan is always ≥ Euclidean.*

WATCH OUT

Scale your features first. If income is in thousands and age is in tens, the large-scale feature dominates every distance. Use `StandardScaler` (mean 0, std 1) before distance-based clustering. (The Mall example below uses two features of similar range, so it is fine there.)

0.3 K-Means in two minutes

K-Means picks k centre points (**centroids**) and repeats two steps until nothing changes:

1. **Assign** every point to its nearest centroid (distance to centroid!).
2. **Update** each centroid to the mean of the points assigned to it.

It is trying to minimise the total squared distance from every point to its own centroid:

$$J = \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2$$

K-Means objective

C_j = the j-th cluster, μ_j = its centroid (the mean of its points), k = number of clusters you must supply. To pick k you plot J against k and look for the **elbow**. Keep this “distance to a centroid” picture in mind. It is both K-Means’ strength and its weakness.

0.4 Where K-Means fails (the reason for everything that follows)

K-Means relies heavily on distance to a centroid. If your data sits in nicely separated round blobs, it works beautifully and creates clusters very easily. But give it slightly different data and it breaks:

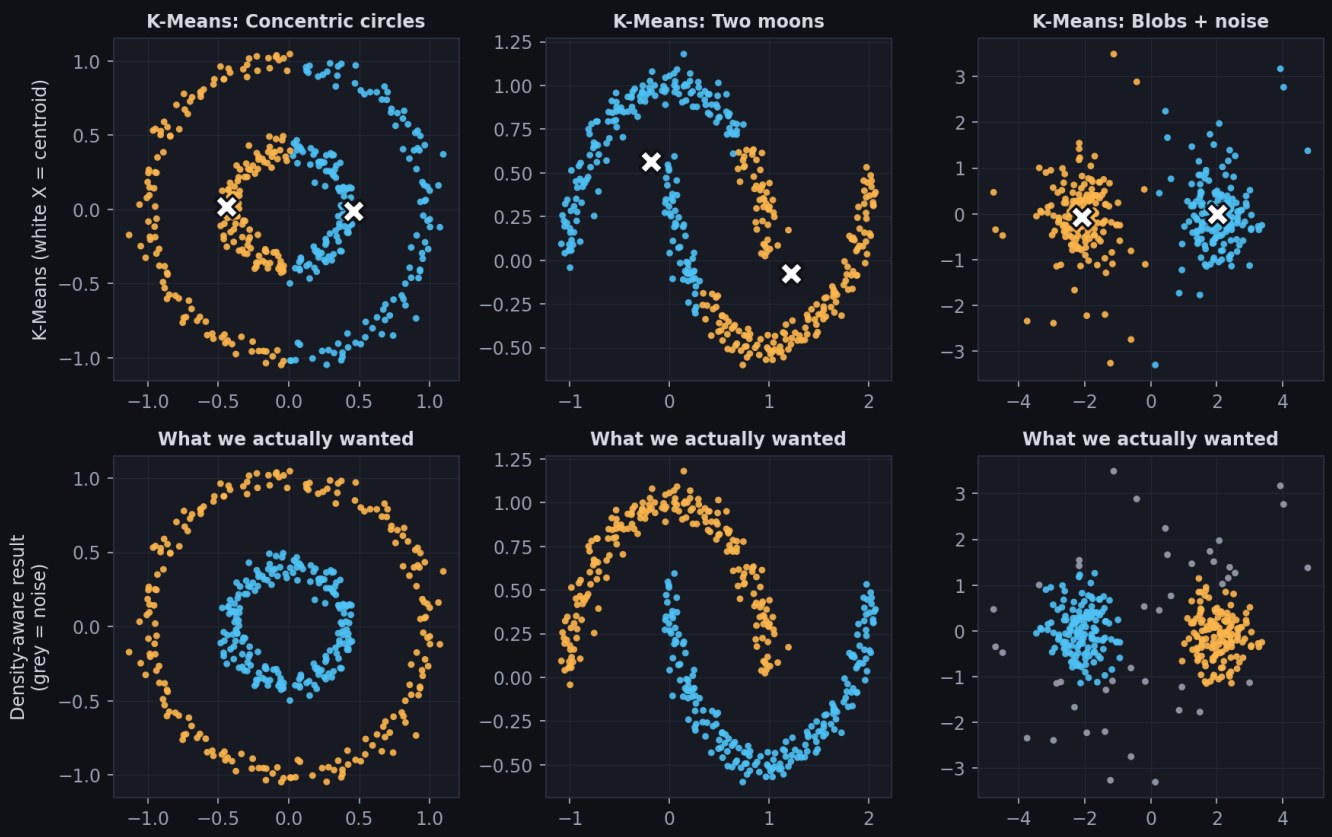


Figure 0.2 – Top row: K-Means (white X = centroid). Bottom row: the grouping we actually wanted, found by a density-aware method.

- **Concentric circles.** We want the inner ring as one cluster and the outer ring as another. K-Means, thinking in terms of centres, simply cuts the plane in half.
- **Two moons (crescents).** The two crescents should be two clusters, but K-Means slices straight through both because its clusters are always round, convex regions.
- **Clusters of different shape or size** (for example three clusters that are not equal-sized blobs): the nearest-centroid rule cannot follow them properly.
- **Noise / outliers.** Some points are pure noise and we would rather not put them in any cluster. K-Means is forced to assign *every* point to some cluster.

> **REMEMBER**
> K-Means gives its best performance on **spherical (circular) clusters that are reasonably separated**. For anything else, we need other tools.

The three problems can be listed precisely (we will meet them again in the DBSCAN chapter):

1. **You must tell it k.** In 10-dimensional data you cannot plot it, and the elbow curve is often ambiguous.
2. **Very sensitive to outliers.** Centroids are means, and one far-away point drags a mean a lot (Figure 0.3).
3. **Centroid-based, so it only finds round clusters.**

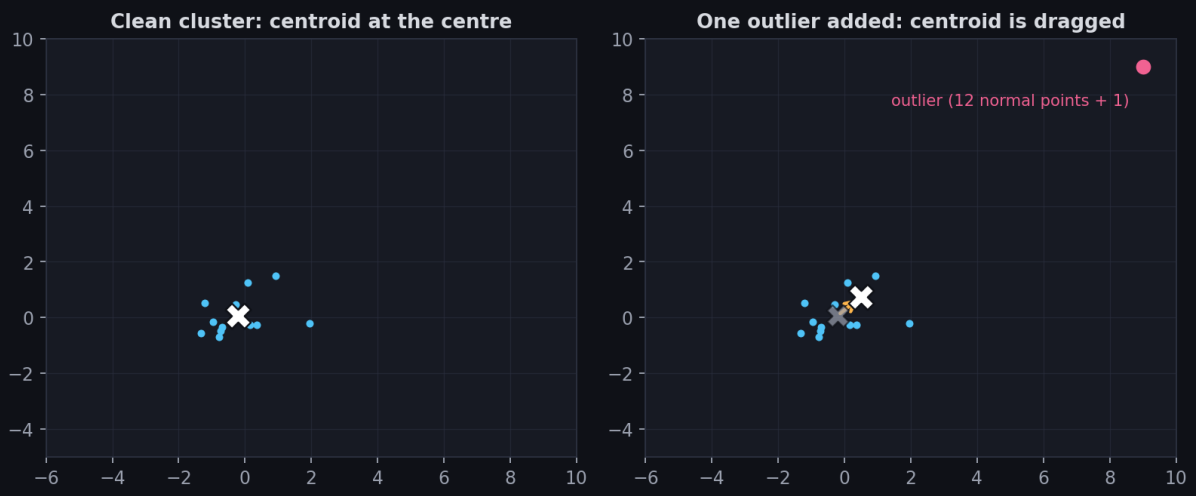


Figure 0.3 – Adding one outlier to a small cluster pulls the centroid (white X) away from where it should be (grey X).

The way out. Machine-learning researchers built other clustering families. We study two:

Family	Algorithm	Idea in one line	Chapter
Hierarchical	Agglomerative / Divisive	Build a *tree* of merges; cut it where you like	1
Density-based	DBSCAN (also OPTICS)	A cluster is a crowded region; sparse points are noise	2

As a teaser, here is what DBSCAN does on a dataset where K-Means struggles: it finds two clusters (orange and blue) and marks the scattered points as noise (black or grey) instead of forcing them into a cluster. We study it properly in Chapter 2.

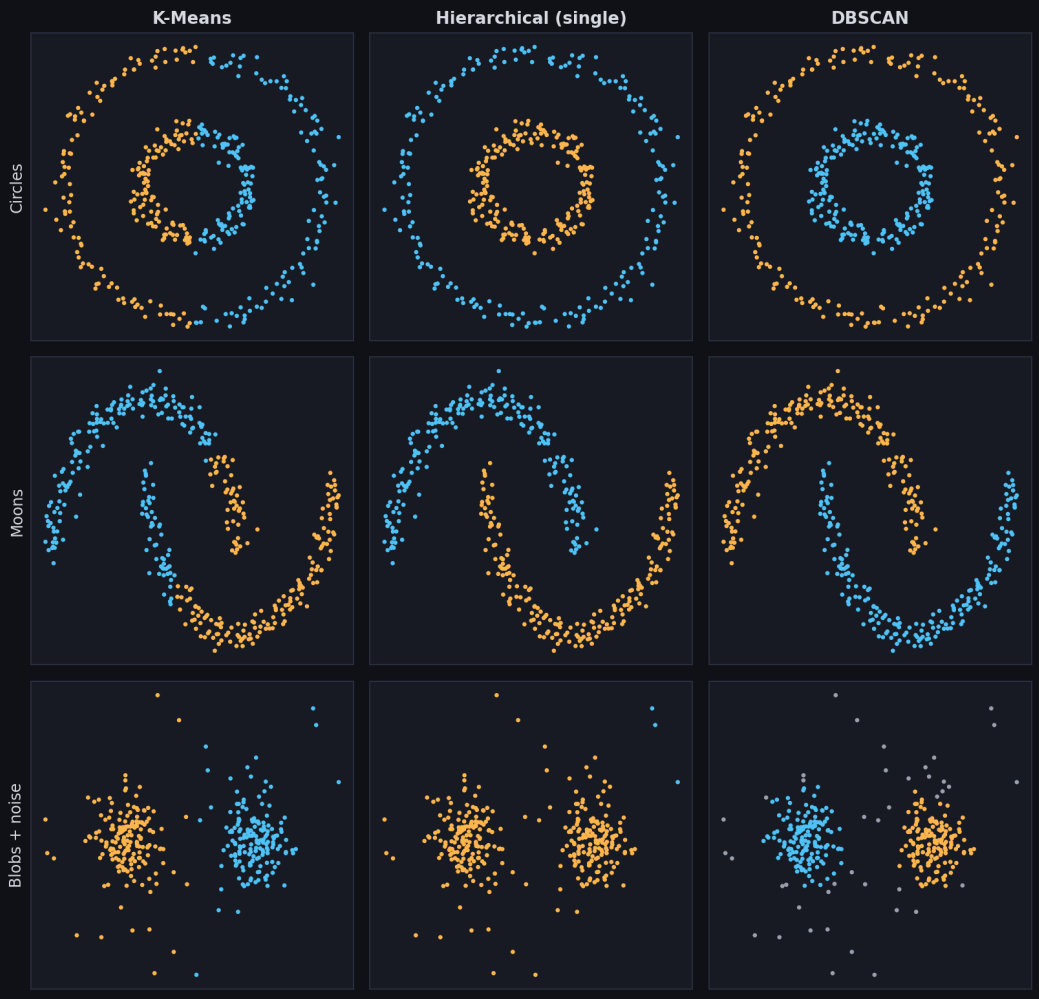


Figure 0.4 – Preview: K-Means vs single-link hierarchical vs DBSCAN on three tricky datasets (grey = noise). No single method wins everywhere.

Chapter 1 · Hierarchical Clustering

In this chapter: why it is needed, the intuition, how the whole algorithm works, its types, which datasets it suits, and a code example. We start with a picture and end with the maths.

1.1 The big idea (Level 1 · Basic)

> **INTUITION**
> Think of a **family tree**, or the biological tree of life: individuals pair up into siblings, siblings into families, families into clans... Hierarchical clustering builds exactly such a tree over your data points. Close points join first; groups of close points join later. **You never have to say "k = 3" in advance.** You build the full tree, and then decide where to cut it.

That tree is called a **dendrogram**, and it also stores the *order* in which things were merged. That is why the method is called *hierarchical*: the tree tells you the hierarchy of the clustering.

1.2 Two flavours

	Agglomerative (bottom-up)	Divisive (top-down)
Start	every point is its own cluster (n clusters)	all points in ONE big cluster
Each step	**merge** the two closest clusters	**split** a cluster into two
End	one single cluster	every point is its own cluster
How common?	Used much more (this chapter's focus)	Rarely used; we only get the idea

> **REMEMBER**
> Agglomerative = **merge**. Divisive = **split**. They are exact opposites.

1.3 Agglomerative clustering, step by step (6 points)

Take six points. We must cluster them. The recipe:

1. Start by assuming **every point is a cluster by itself**: 6 points → 6 clusters.
2. Find the **two closest** clusters, **merge** them into one, and **write down** this merge somewhere.
3. Repeat until only one cluster is left.

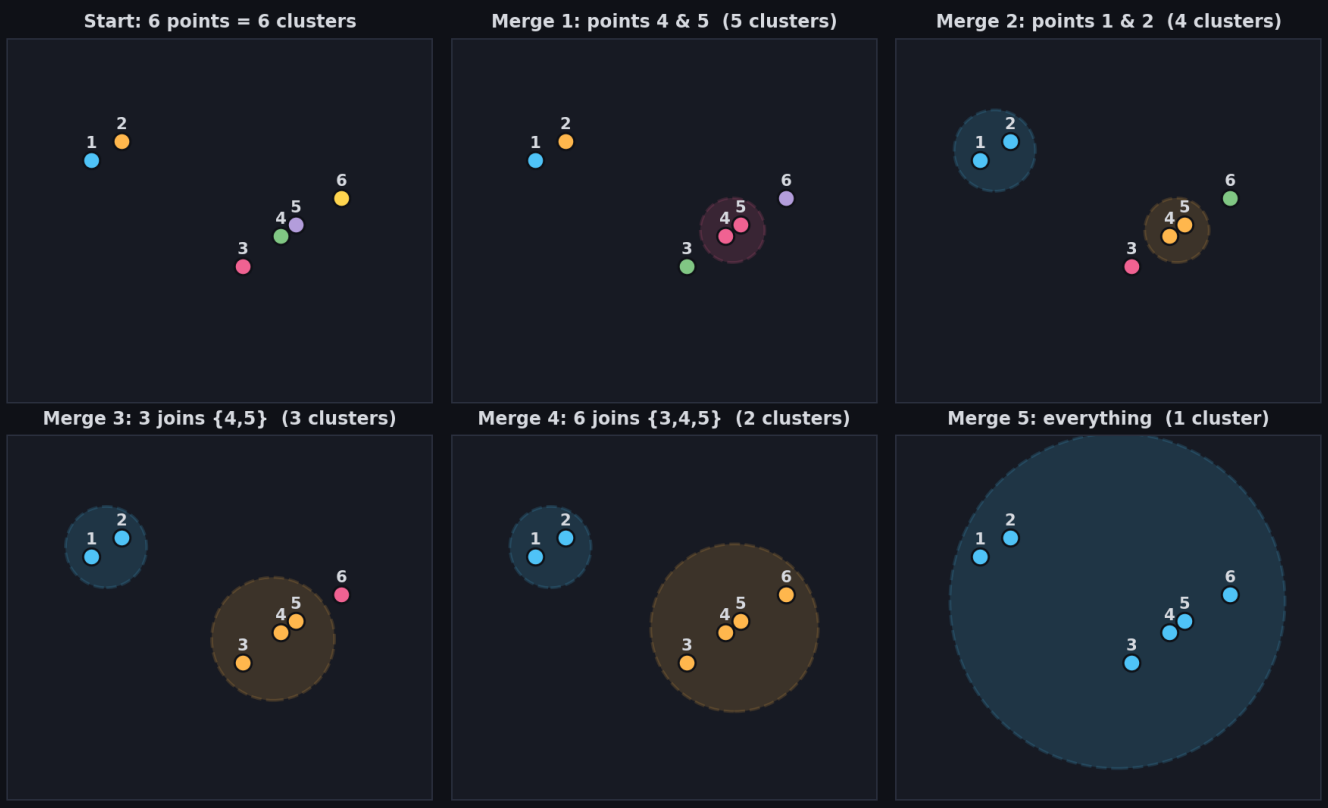


Figure 1.1 – The six panels run left to right, top to bottom. A dashed circle marks a merged cluster.

Walking through the picture:

- **Merge 1:** points 4 and 5 are the closest pair, so they become one cluster (we record this merge). Clusters: 6 → **5**.
- **Merge 2:** points 1 and 2 are also very close, so they merge too (record it). Clusters: **4**.
- **Merge 3:** point 3 is closest to the cluster {4, 5} (not to {1, 2}), so it joins {4, 5}, forming {3, 4, 5}. Clusters: **3**.
- **Merge 4:** point 6 is very close to {3, 4, 5}, so it joins. Clusters: **2**: {1, 2} and {3, 4, 5, 6}.
- **Merge 5:** finally those two merge, leaving **1** big cluster.

In the lecture the clusters were labelled C1, C2, C3, C4 as they appeared. The counts go **6 → 5 → 4 → 3 → 2 → 1**. With n points there are always exactly $n - 1$ merges.

$$\text{number of merges} = n - 1 \qquad \text{here } 6 - 1 = 5$$

1.4 The dendrogram: your record of every merge

Every time we merged, we recorded it. Draw all those records together and you get a tree: the **dendrogram**. Leaves at the bottom are the points; each horizontal bar is a merge; the **height** of a bar is the distance at which the merge happened.

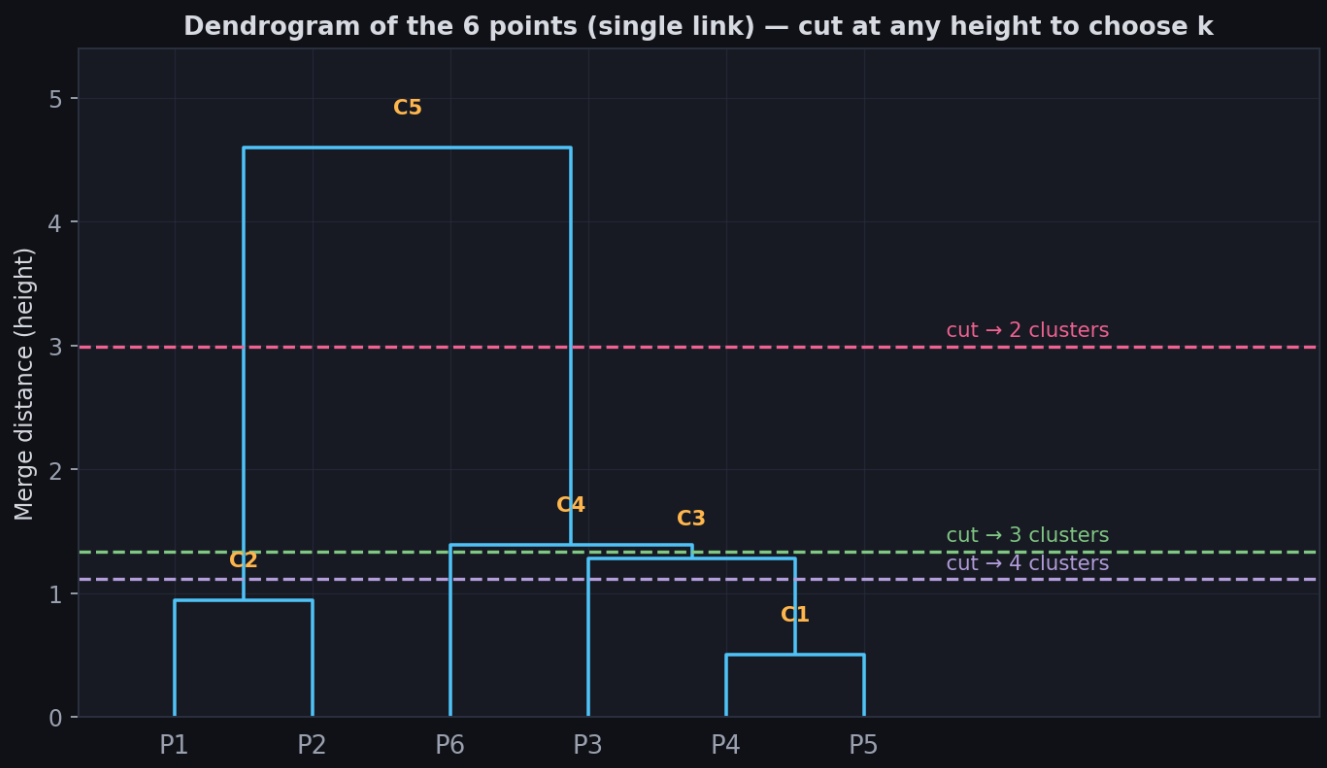


Figure 1.2 – The dendrogram of the six points. C1…C5 are the merges in order; dashed lines show where cutting gives 2, 3 or 4 clusters.

Reading it is immediate: 4 and 5 were closest (lowest bar), 1 and 2 were also close, 3 was close to {4, 5}, 6 was close to {3, 4, 5}, and the two big groups joined last, at the greatest distance.

Getting any number of clusters by cutting

At the very end everything is one cluster, but you can **cut the tree with a horizontal line** and keep whatever hangs below it:

Cut at	Clusters you get	Which are
very top (above all merges)	1	everything
below the last merge (the cut “from here” for 2)	2	{1, 2} and {3, 4, 5, 6}
below the 2nd-last merge	3	{1, 2}, {3, 4, 5}, {6} (point 6 separately)
below the 3rd-last merge	4	{1, 2}, {3}, {4, 5}, {6}
at the very bottom	6	every point alone

REMEMBER

One tree, every possible clustering. You can stop at any stage and keep that many clusters. The whole hierarchy is stored in the dendrogram, so K-Means’ question “what is k?” can be answered *after* clustering by looking at the tree.

1.5 Divisive clustering (the opposite direction)

Same six points, reversed film. Start with **one** cluster holding everything. Split it into two, then split one of those further (3 clusters), then 4, and so on, until every point is alone (6 clusters).



Figure 1.3 – Divisive clustering is the agglomerative movie played backwards (same final tree).

Why is agglomerative used more? Merging is easier than breaking: to merge you only compare pairs of existing clusters by distance. To split you must decide *how* to cut a cluster into two parts, and the number of possible ways explodes:

$$\text{ways to split } n \text{ points into 2 non-empty groups} = 2^{n-1} - 1$$

why splitting is hard

For n = 6 that is $2^5 - 1 = 31$ ways. For n = 30 it is already about 5.4×10^8 ways. Practical divisive methods (like DIANA, or repeatedly running K-Means with k = 2, known as *bisecting K-Means*) must use shortcuts. Merging needs at most n(n−1)/2 pair-distances.

1.6 The exact algorithm (Level 2 · Intermediate)

If you wanted to code agglomerative clustering yourself, you would write a loop of just a few steps. The central data structure is the **proximity matrix**.

> **MATH**
> **Proximity matrix** = an $n \times n$ table where entry (i, j) is the distance between point i and point j. The diagonal is 0 (a point's distance to itself) and the table is symmetric, so d(i, j) = d(j, i).

The algorithm

1. **Compute** the proximity matrix (all pairwise distances). Treat every point as a cluster.
2. **Loop:** find the two closest clusters and **merge** them.
3. **Update** the proximity matrix: the merged cluster's row and column must be recomputed (its distance to every other cluster), the rest stay the same.
4. **Repeat** until only one cluster (a 1 × 1 matrix) remains. Record every merge: that is the dendrogram.

A worked example with real numbers (single link)

Five points: P1 (1, 1), P2 (1.5, 2), P3 (6, 5), P4 (6.5, 4.5), P5 (5, 3). We use “distance between clusters = distance between their *closest* points” (single link; the choice is explained in 1.7).



Figure 1.4 – The five points and the dendrogram we are about to build by hand.

	P1	P2	P3	P4	P5
P1	0	1.12	6.40	6.52	4.47
P2	1.12	0	5.41	5.59	3.64
P3	6.40	5.41	0	0.71	2.24
P4	6.52	5.59	0.71	0	2.12
P5	4.47	3.64	2.24	2.12	0

Initial matrix: all 5 points. Smallest off-diagonal entry is d(P3, P4) = 0.71 → merge them into C1.

	P1	P2	C1	P5
P1	0	1.12	6.40	4.47
P2	1.12	0	5.41	3.64
C1	6.40	5.41	0	2.12
P5	4.47	3.64	2.12	0

After merge 1 (4×4). Rows/columns for C1 were recomputed with the MIN rule. Smallest is d(P1, P2) = 1.12 → C2.

	C2	C1	P5
C2	0	5.41	3.64
C1	5.41	0	2.12
P5	3.64	2.12	0

After merge 2 (3×3). Smallest is d(P5, C1) = 2.12 → merge into C3.

	C2	C3
C2	0	3.64
C3	3.64	0

After merge 3 (2×2). The last entry d(C2, C3) = 3.64 → final merge.

EXAMPLE

How an update works. After merging P3 and P4 into C1, the distance from P5 to C1 is min(d(P5, P3), d(P5, P4)) = min(2.24, 2.12) = **2.12**. Other entries (like P1–P2) are unchanged. With a different linkage the same cell would be computed differently (max → 2.24, average → 2.18).

The tricky part is exactly this update: once clusters contain several points, what is the distance between two clusters? Everything depends on that definition, and it is what separates the different

types of agglomerative clustering. We cover them next.

Try it: the algorithm from scratch in Python

About 25 lines of NumPy. It is deliberately naive so every line maps to one step above.

agglomerative_scratch.py

```
import numpy as np

def agglomerative(X, linkage="single"):
    """Naive agglomerative clustering. Returns the list of merges."""
    n = len(X)
    # Step 0: proximity matrix (n x n) of Euclidean distances
    D = np.linalg.norm(X[:, None, :] - X[None, :, :], axis=2)
    clusters = {i: [i] for i in range(n)}      # every point starts as its own cluster
    merges = []
    while len(clusters) > 1:
        ids = list(clusters)
        best = (np.inf, None, None)
        for a in range(len(ids)):               # find the two closest clusters
            for b in range(a + 1, len(ids)):
                A, B = clusters[ids[a]], clusters[ids[b]]
                d = D[np.ix_(A, B)]             # all |A| x |B| point-to-point distances
                dist = {"single": d.min(), "complete": d.max(), "average": d.mean()}[linkage]
                if dist < best[0]:
                    best = (dist, ids[a], ids[b])
        dist, i, j = best
        merges.append((clusters[i][:], clusters[j][:], round(float(dist), 3)))
        clusters[i] = clusters[i] + clusters.pop(j)   # merge j into i and record it
    return merges
```

```
([2], [3], 0.707)
([0], [1], 1.118)
([2, 3], [4], 2.121)
([0, 1], [2, 3, 4], 3.64)
```

These four merges and heights match the hand calculation and SciPy's `linkage` output exactly. (Point indices start at 0 in code, so [2],[3] means P3, P4.)

1.7 Four ways to measure distance between clusters (linkage)

Distance between two *points* is easy. Distance between two *clusters* (groups of points) can be defined in several ways. Each way is a **linkage**, and each gives a different type of agglomerative clustering: **Single (MIN), Complete (MAX), Average, Ward.** The lecture calls this the *inter-cluster similarity*.

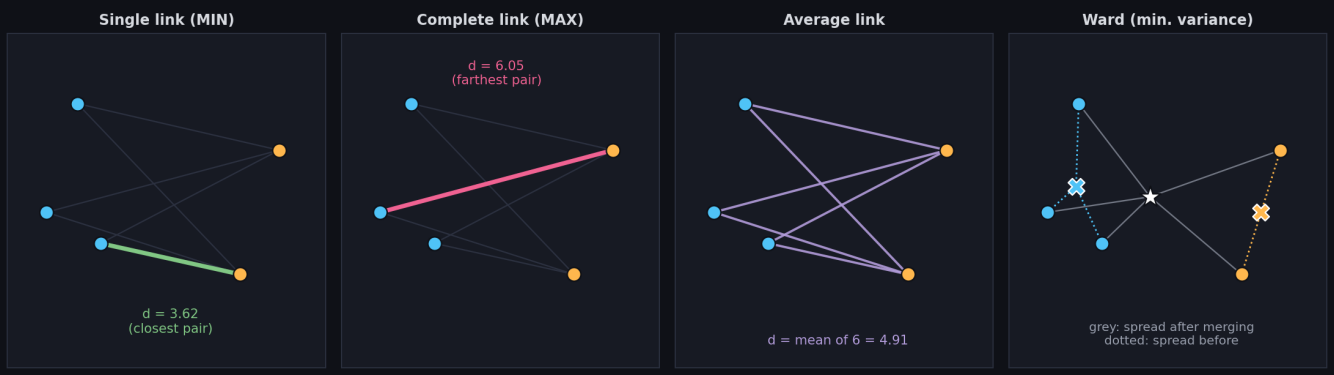


Figure 1.5 – Two clusters (3 blue + 2 orange points). The lines show which distances each linkage uses.

In the figure, two clusters have 3 × 2 = **6** point-to-point distances. The linkages differ only in what they do with those six numbers:

(a) Single link, also called MIN

$$d(A,B) = \min_{a \in A,\ b \in B} d(a,b)$$ Single / MIN

Use the **closest pair** across the two clusters (here 3.62). Two clusters are "close" if even one pair of their points is close.

Advantage: when clusters have a clear **gap** between them, single link separates them correctly, and it can follow non-round shapes. **Disadvantage:** it is **sensitive to outliers and noise**: a thin bridge of noisy points chains two clusters together (the "chaining effect"). Use it only when your data is cleanly separated.

(b) Complete link, also called MAX

$$d(A,B) = \max_{a \in A,\ b \in B} d(a,b)$$ Complete / MAX

Use the **farthest pair** (here 6.05). Two clusters are close only if *all* their points are fairly close, so every cluster stays compact.

Advantage: outliers and noise are controlled very well. **Disadvantage:** if one cluster is large and another small, the **big cluster tends to get broken** into pieces, which is not desirable.

(c) Group average

$$d(A,B) = \frac{1}{|A|\,|B|} \sum_{a \in A} \sum_{b \in B} d(a,b)$$ Average

Compute **all** |A|×|B| distances (d1, d2, …, d6), add them up, divide by |A|·|B| = 3 × 2 = 6. In the figure: average = 4.91. This sits **between MIN and MAX**, so its good and bad effects also sit in between. It balances the shortcomings of both.

(d) Ward's method (the default in scikit-learn)

Ward asks a different question: *"If I merge these two clusters, how much does the total spread (variance) inside clusters increase?"* It always picks the merge that increases it the **least**. In words, the lecture's recipe is: find the centroid of the merged points, measure distances from every point to its centroid, and subtract the within-cluster spreads that already existed. The key phrase is **"minimise the variance"**.

$$SSE(C) = \sum_{x \in C} \|x - \mu_C\|^2$$ spread of one cluster

$$\Delta(A,B) = SSE(A \cup B) - SSE(A) - SSE(B) = \frac{|A|\,|B|}{|A| + |B|} \|\mu_A - \mu_B\|^2$$ Ward distance

μ_A, μ_B are the centroids of the two clusters and |A|, |B| their sizes. The two forms are equal: the left one is "increase in total squared error caused by merging", the right one is a shortcut that only needs centroids and sizes.

EXAMPLE

Tiny numeric check (1-D). A = {1, 2}, B = {5, 7}. $\mu_A = 1.5$, $\mu_B = 6$. SSE(A) = 0.25 + 0.25 = 0.5; SSE(B) = 1 + 1 = 2. Merged {1, 2, 5, 7} has mean 3.75 and SSE = 7.5625 + 3.0625 + 1.5625 + 10.5625 = 22.75. So $\Delta = 22.75 - 0.5 - 2 =$ **20.25**. Shortcut: $(2 \cdot 2)/(2+2) \times (6 - 1.5)^2 = 1 \times 20.25 =$ **20.25**. ✓

CORRECTION / UPDATE

Note on the spoken Ward description. The lecture explains Ward by summing squared distances to the centroid and subtracting the clusters' internal variances. That is the same idea; the clean formula above (increase in SSE) is how textbooks and scikit-learn define it. Ward only makes sense with **Euclidean** distance.

Same data, four linkages. On the 5-point example the heights at which merges happen are:

Merge #	Single	Complete	Average	Ward
1	0.707	0.707	0.707	0.707
2	1.118	1.118	1.118	1.118
3	2.121	2.236	2.179	2.483
4	3.640	6.519	5.339	8.215

The first merges agree (P3–P4, P1–P2 are simply the closest pairs), but the later heights differ a lot, because the rules for "distance between groups" differ. That is also why the final tree shape can differ between linkages on harder data.

Summary of the four linkages

Linkage	Cluster distance	Good at	Weak at	Cluster shape
Single (MIN)	closest pair	clear gaps; elongated shapes	outliers / noise → chaining	long, stringy
Complete (MAX)	farthest pair	controlling outliers	breaks big clusters	compact, equal diameter
Average	mean of all pairs	a balance of the two	a bit slower to explain	in between
Ward	increase in variance	compact, similar-size blobs; default	needs Euclidean; blob-biased	round, equal size

1.8 The four linkages on real shapes

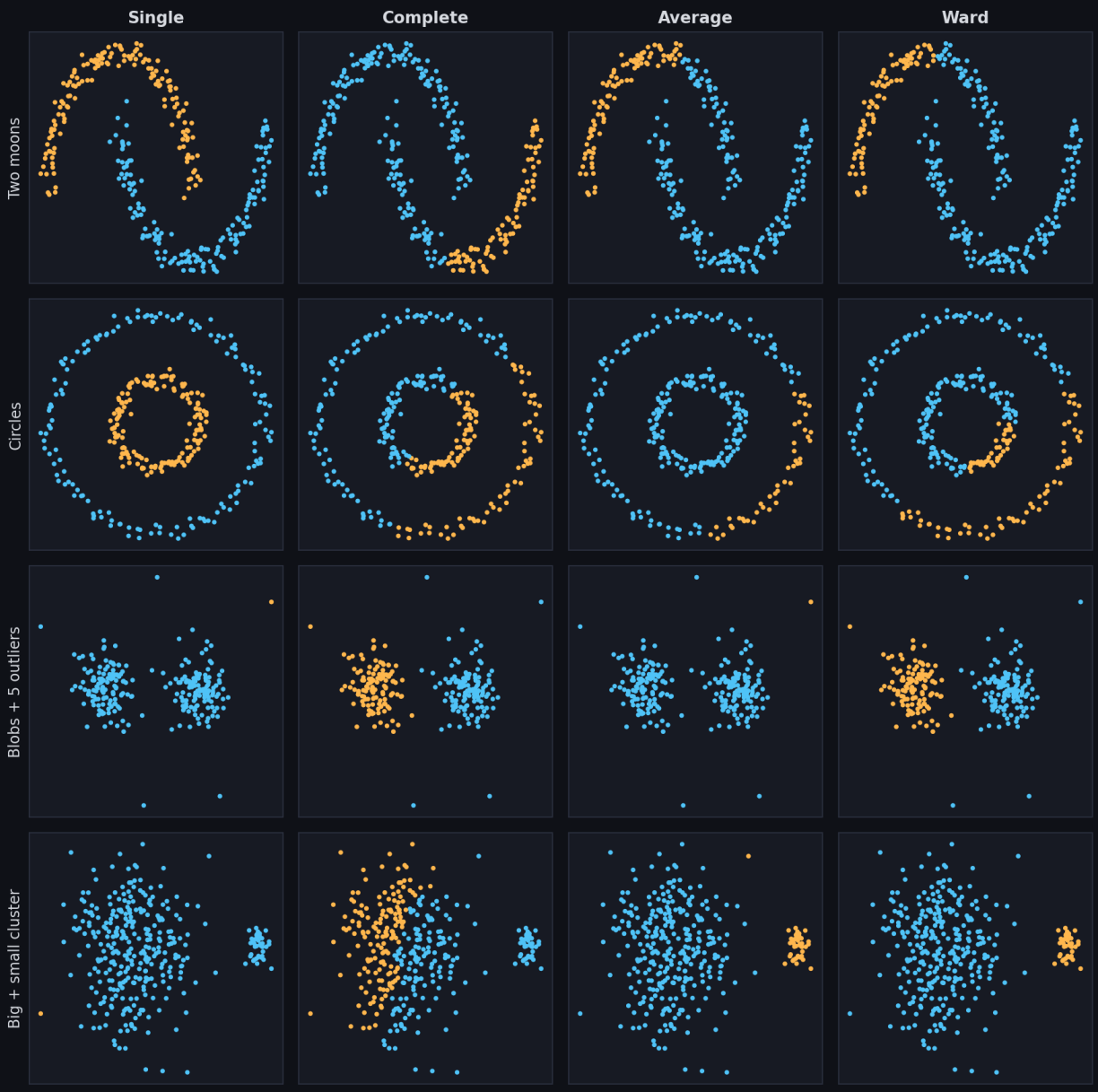


Figure 1.6 – Rows: datasets. Columns: linkage. Colours = the 2 clusters found (all runs ask for n_clusters = 2).

- **Two moons:** only **single link** follows the crescents. The others cut by compactness.
- **Circles:** again only single link separates the rings; Ward and complete cut the plane.
- **Blobs + 5 outliers:** single link spends its clusters on isolated outliers, which is the lecture's warning about noise. Complete, average and Ward handle it.
- **Big + small cluster:** look how the methods split the big cloud. This is the "breaks the big cluster" behaviour of the farthest-point rule.

> **PRO TIP**
> No linkage is best everywhere. A good habit: run Ward first (it is the default for a reason), then try average and single and compare. If shapes are very irregular or noisy, switch to DBSCAN (Chapter 2).

1.9 How many clusters? Reading the dendrogram

In K-Means you had to tell the algorithm k, using the elbow method. Here too we do not know k in advance, and with high-dimensional data we cannot plot and look. So we use the **dendrogram**.

REMEMBER

The rule: find the **longest vertical line that is not cut by any horizontal line**. Cut the dendrogram through the middle of that line. The number of vertical lines the cut crosses = the number of clusters.

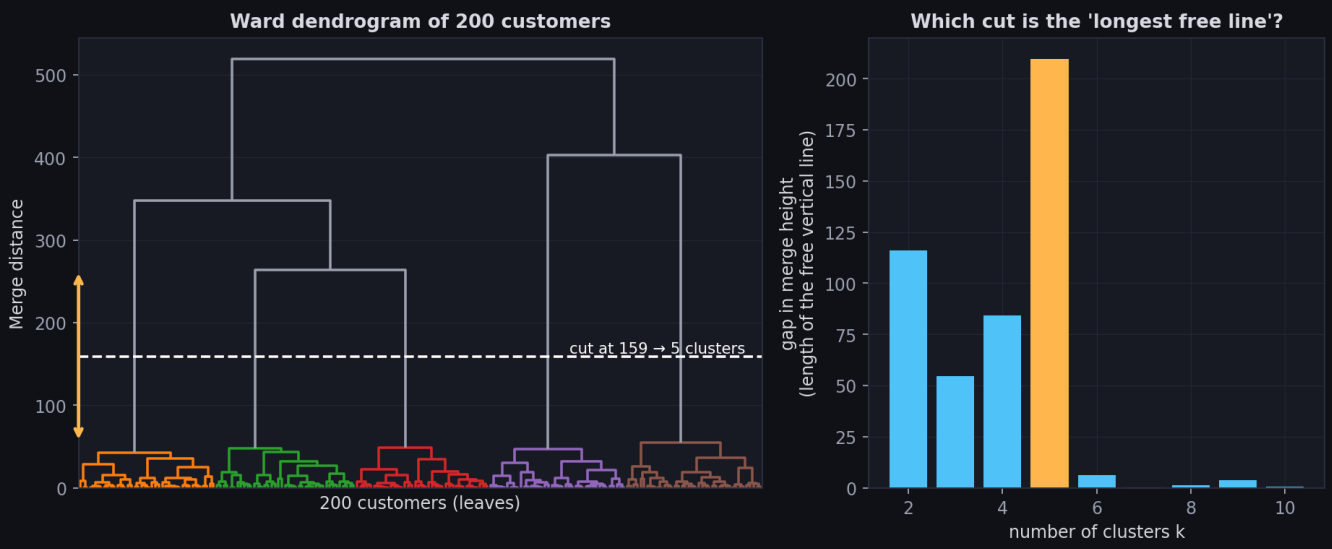


Figure 1.7 – Left: dendrogram of 200 customers with the cut (dashed). Right: the same rule as numbers: the biggest gap between consecutive merge heights is at k = 5.

Why this works. A merge height is the distance between the two clusters being joined (the vertical axis = *inter-cluster distance*). A very long vertical line means "these clusters were far apart, yet we were forced to merge them at last". So just below that line, the groups are well separated. Lines that are cut by a horizontal bar are short pieces and carry no such signal: ignore them.

$$k^* = \text{argmax}_k \left(h_{n-k} - h_{n-k-1}\right)$$

the 'longest free line' as a formula

h_m = height of the m-th merge, sorted ascending (n − 1 merges in total). The difference $h_{n-k} - h_{n-k-1}$ is the length of the free vertical line when exactly k clusters remain. In Figure 1.7 the gap for k = 5 is the tallest bar.

Second example from the lecture: another dendrogram where the longest free vertical line, extended into a horizontal cut, crosses **5** vertical lines, so that data has 5 clusters. Practise by drawing a dendrogram, covering the top, and finding the tallest uninterrupted stick.

WATCH OUT

The rule is a *heuristic*. At the very top of a tree gaps are naturally large (merging two huge groups), so k = 2 can win spuriously. Cross-check with domain knowledge or a score such as the silhouette (Section 1.13).

1.10 scikit-learn: the AgglomerativeClustering class

The class has four important parameters:

Parameter	Meaning	Options / default
`n_clusters`	How many clusters to form (where to cut the tree).	int, default 2. Must be None if distance_threshold is used.

Parameter	Meaning	Options / default
`affinity` → `metric`	Which **distance metric** is used between points: euclidean, manhattan, cosine, L1, L2…	default 'euclidean'. Ward only allows euclidean.
`linkage`	The type of agglomerative clustering from 1.7: **ward** (default), **complete**, **average**, **single**.	default 'ward'
`distance_threshold`	A cut height. Clusters whose merge distance would be above it are **not merged**.	default None. Then n_clusters=None, compute_full_tree=True.

> **CORRECTION / UPDATE**
> **Library update:** in the video the parameter is called `affinity`. In scikit-learn ≥ 1.2 it was renamed `metric` and `affinity` was removed in 1.4. On a recent version write `metric='euclidean'`. The meaning is identical.

Useful extras: `compute_distances=True` stores merge heights in `distances_` so you can draw a dendrogram from the fitted model; `connectivity` lets you restrict merges to neighbours (e.g. image pixels or a graph). After fitting: `labels_`, `n_clusters_`, `children_`.

1.11 Hands-on project: Mall customers

The data: **200 customers** of a mall with *Age*, *Gender*, *Annual Income (k$)* and *Spending Score (1–100)*. We use just two columns for every customer: **Annual Income** and **Spending Score**. (Two features = easy to draw, so we can check the result by eye.)

Step 1 – dendrogram with SciPy, to decide k

mall_step1.py

```
import pandas as pd
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch

df = pd.read_csv("Mall_Customers.csv")
X = df.iloc[:, [3, 4]].values            # Annual Income, Spending Score

plt.figure(figsize=(10, 5))
dend = sch.dendrogram(sch.linkage(X, method="ward"))
plt.title("Dendrogram"); plt.xlabel("Customers"); plt.ylabel("Euclidean distance")
plt.show()
```

Apply the rule of 1.9 to the picture: the longest free vertical line is cut by a horizontal line crossing **5** branches, so we choose **k = 5**.

Step 2 – fit the model and get labels

mall_step2.py

```
from sklearn.cluster import AgglomerativeClustering

hc = AgglomerativeClustering(n_clusters=5, metric="euclidean", linkage="ward")
labels = hc.fit_predict(X)          # one label (0-4) for each of the 200 customers

plt.figure(figsize=(7, 5))
for c in range(5):
    plt.scatter(X[labels == c, 0], X[labels == c, 1], s=30, label=f"Cluster {c}")
plt.xlabel("Annual income (k$)"); plt.ylabel("Spending score (1-100)")
plt.legend(); plt.show()
```

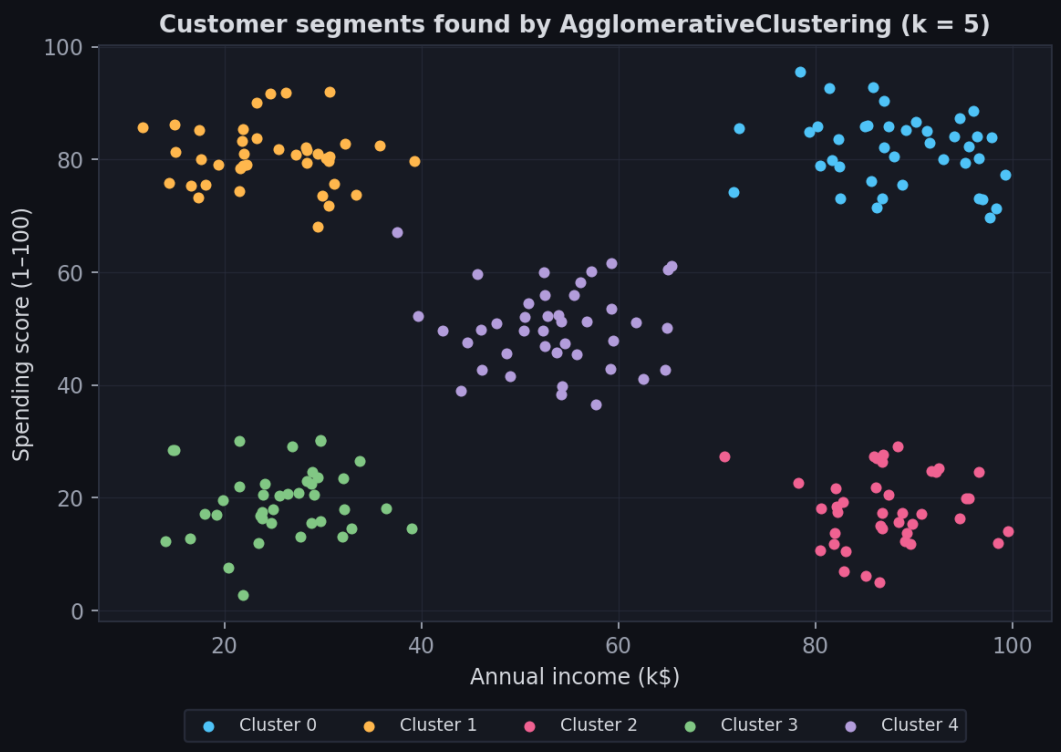


Figure 1.8 – Five customer segments (synthetic stand-in with the same structure as the real Mall data).

Reading the groups: low income + low spending, low income + high spending, middle income + middle spending, high income + low spending, and high income + high spending. A marketer would now design a different offer for each, for example loyalty rewards for the high-income, low-spending group.

> **EXAMPLE**
> **Assignment from the lecture.** Find a dataset of countries with GDP, population and surface area. Standardise the columns, run agglomerative clustering, and find **which countries fall in the same cluster as India**. Then read the dendrogram: which country merges with India first = its nearest neighbour on those features. (Great use of the dendrogram's "who is closest to whom" information.)

1.12 Strengths and limitations

Benefits

1. **Widely applicable.** K-Means works only on round blobs, but because hierarchical clustering has several techniques (MIN, MAX, average, Ward), choosing them differently lets you cluster very different shapes of data correctly (Figure 1.6).
2. **You get the dendrogram.** At every stage you know which point or group is closest to which. In K-Means you only get centroids; "who is this point's nearest neighbour?" is not answered, since all points in a cluster look equal. With the hierarchy you know the exact nearest relative of every point. So when you need to find *very close things* (nearest country, similar customers, similar genes), this algorithm is a good choice.

Limitation: it does not scale to large datasets

The algorithm builds the $n \times n$ proximity matrix and keeps it in RAM. If you have $n = 10^6$ (one million) points, the matrix has $n^2 = 10^{12}$ entries.

$$\text{memory} \approx n^2 \times 8 \text{ bytes} = (10^6)^2 \times 8 = 8 \times 10^{12} \text{ bytes} \approx 8 \text{ TB} \quad \text{memory estimate}$$

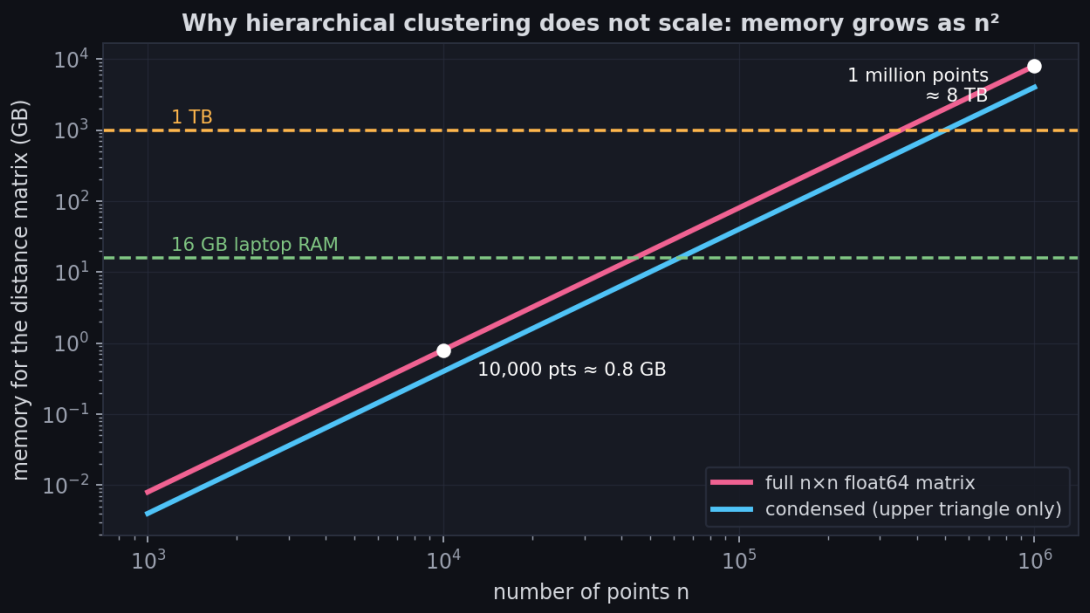


Figure 1.9 – Memory for the distance matrix grows with n². Even the half-size "condensed" matrix is far beyond normal RAM at one million points.

CORRECTION / UPDATE

Number check. The lecture says 10^{12} entries is "around 10 GB". The conclusion (you cannot fit it in RAM) is right, but the size is much larger: 10^{12} numbers × 8 bytes ≈ 8 TB (about 4 TB if you store only half the symmetric matrix). Even 100,000 points need ≈ 80 GB.

Quantity	Cost
Memory (proximity matrix)	$O(n^2)$
Time, naive loop (as in our code)	$O(n^3)$
Time with a priority queue / nearest-neighbour chain	$O(n^2 \log n)$ in general; $O(n^2)$ for single link (SLINK) and Ward (NN-chain)

PRO TIP

Working around the size limit: (1) cluster a random sample, then assign the rest to the nearest cluster; (2) run K-Means with a large k first (say 1000), then hierarchically cluster the centroids; (3) use BIRCH (a tree-based scalable method); (4) pass a sparse `connectivity` matrix (e.g. 10 nearest neighbours) so far fewer pairs are considered.

1.13 Pro level (Level 3)

Lance–Williams: one update formula for all linkages

Recomputing the distance between a new merged cluster (i ∪ j) and any other cluster k does **not** require going back to the raw points. All linkages can be written with one formula that uses only the old distances, which is how fast implementations update the proximity matrix:

$$d(k,\, i \cup j) = \alpha_i\, d(k,i) + \alpha_j\, d(k,j) + \beta\, d(i,j) + \gamma\, |d(k,i) - d(k,j)| \qquad \text{Lance–Williams}$$

Linkage	α_i	α_j	β	γ
Single	1/2	1/2	0	−1/2

Linkage	α_i	α_j	β	γ
Complete	1/2	1/2	0	+1/2
Average	|i| / (|i|+|j|)	|j| / (|i|+|j|)	0	0
Ward (on squared distances)	(|i|+|k|) / (|i|+|j|+|k|)	(|j|+|k|) / (|i|+|j|+|k|)	−|k| / (|i|+|j|+|k|)	0

Check with single link: ½d(k,i) + ½d(k,j) − ½|d(k,i) − d(k,j)| = min(d(k,i), d(k,j)). ✓ That is exactly the MIN rule used in our worked example.

The SciPy linkage matrix

`Z = linkage(X, 'ward')` returns an (n−1) × 4 array. Row *t* says: "at step t, clusters Z[t,0] and Z[t,1] merged at distance Z[t,2], creating a cluster with Z[t,3] points." Original points are numbered 0…n−1; the new cluster created at step t is numbered n + t. `fcluster(Z, t=5, criterion='maxclust')` cuts the tree into 5 clusters; `criterion='distance'` cuts at a height.

How good is the tree? Cophenetic correlation

The **cophenetic distance** of two points is the height at which they first end up in the same cluster. If the dendrogram is a faithful summary of the data, cophenetic distances should correlate with the original distances:

$$c = \frac{\sum_{i<j}(x_{ij}-\bar{x})(t_{ij}-\bar{t})}{\sqrt{\sum_{i<j}(x_{ij}-\bar{x})^2 \sum_{i<j}(t_{ij}-\bar{t})^2}}$$

cophenetic correlation

x_{ij} = original distance of points i and j, t_{ij} = cophenetic distance, bars = averages. Close to 1 = the tree preserves the distances well. In code: `c, _ = cophenet(Z, pdist(X))`.

Evaluating and comparing results

$$s(i) = \frac{b(i)-a(i)}{\max\{a(i),\, b(i)\}} \in [-1, 1]$$

silhouette of one point

a(i) = average distance from point i to the other points of its own cluster; *b(i)* = average distance to the points of the *nearest other* cluster. Near +1 = well inside its cluster; near 0 = on a boundary; negative = probably in the wrong cluster. The mean silhouette over all points lets you compare different k or different linkages (`sklearn.metrics.silhouette_score`).

WATCH OUT

Inversions. With centroid or median linkage, a later merge can happen at a *lower* height than an earlier one, so the dendrogram has crossing "inverted" bars. Single, complete, average and Ward never do this. That is a good reason they are the standard four.

PRO TIP

Real-project checklist: scale features → pick metric (cosine for text/embeddings, Euclidean otherwise) → try Ward, average, single → draw the dendrogram (truncate with `dendrogram(Z, truncate_mode='lastp', p=20)` for large n) → pick k by the longest free line AND by silhouette → inspect cluster profiles (means per cluster) → name the segments.

Chapter 1 summary

- Hierarchical clustering builds a tree of merges (agglomerative) or splits (divisive); you choose k afterwards by cutting.
- The algorithm = proximity matrix + repeat {merge closest, update matrix}. The only design choice is the cluster-to-cluster distance (linkage).
- Single = closest pair (shapes, but noise-sensitive); Complete = farthest pair (noise-robust, breaks big clusters); Average = in between; Ward = minimise variance increase (default).
- Cut at the longest vertical line that no horizontal line crosses. The vertical axis is inter-cluster distance.
- Strong: flexible, gives the full hierarchy. Weak: $O(n^2)$ memory, so not for large data.

Chapter 2 · DBSCAN

DBSCAN is a very famous machine-learning clustering algorithm and is asked a lot in interviews. Plan of this chapter: first the **geometric intuition**, then a **small Python code**, and finally a **visualisation tool** that makes the intuition solid.

2.1 Why do we need yet another algorithm?

K-Means is a good algorithm, but it has flaws, and some of them are important enough that a new method was required. Three of them:

Flaw 1 – you must specify k. Before clustering you must say how many clusters to form. Imagine 10-dimensional data: you cannot plot it, so you cannot even see how many clusters there should be. The elbow method helps, but the elbow curve is often ambiguous and the right value is not clear.

Flaw 2 – very sensitive to outliers. K-Means keeps calculating distances from every centroid to every point. One outlier drags the centroids out of the centres of their clusters (recall Figure 0.3).

Flaw 3 – it is centroid-based. All of its mechanics revolve around centroids, so on weirdly shaped, non-spherical data it fails and clusters in a completely wrong way (Figure 0.2).

Because of these three reasons the community realised K-Means alone is not enough: it performs well in some scenarios but fails in many others. DBSCAN fulfilled that new requirement.

K-Means problem	How DBSCAN answers it
Must choose k	Finds the number of clusters by itself (you choose ε and MinPts instead)
Outliers shift centroids	Has no centroids; outliers are simply labelled **noise** (−1)
Only round clusters	Cluster shape is free: a cluster is a connected crowded region

2.2 Density-based clustering: the idea

K-Means is **centroid-based**: create centroids, calculate distances, move points, make new clusters. DBSCAN follows a completely different style: **density-based clustering**. You cluster the data using the *density* of points: how crowded a region is.

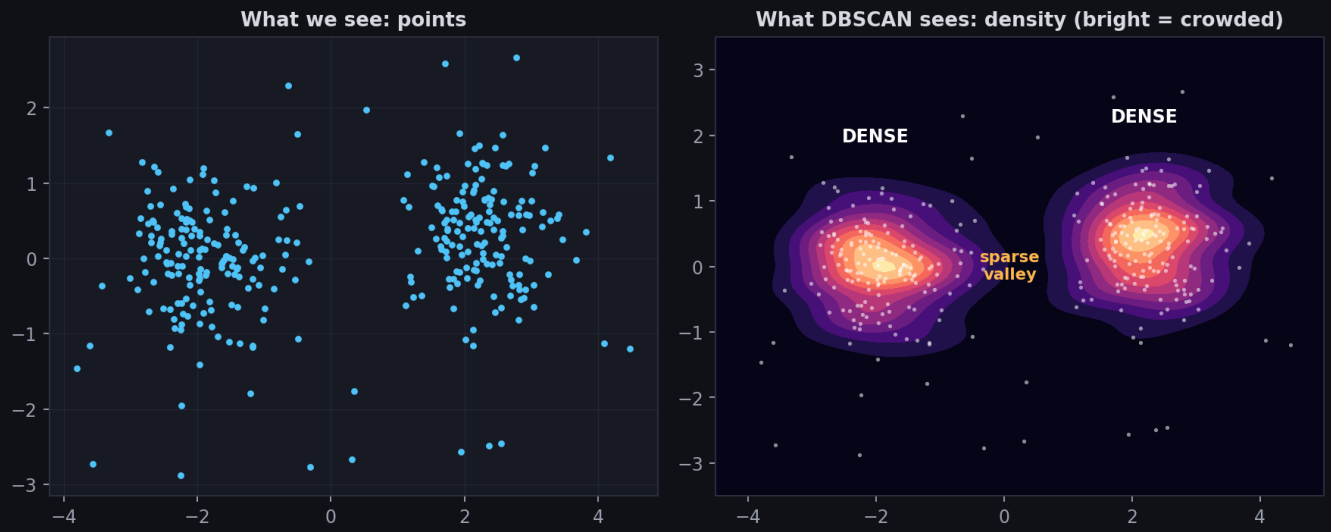


Figure 2.1 – Left: raw points. Right: a density map. Two dense regions are separated by a sparse valley, and DBSCAN uses that sparse valley to separate two clusters.

Look at the picture the way DBSCAN does: this is a **dense** region (many points close together), this is a **sparse** region (few points), and here is a dense region again. The sparse region lets DBSCAN cut two dense regions apart into different clusters. The scattered points living in the sparse area are not important to any cluster: they are **noise**.

> **INTUITION**
> **Cities from space at night.** Bright clusters of lights = dense regions = cities (clusters). Dark gaps between = sparse = no cluster. A lonely farmhouse light far from everything = noise. Nobody needs to say "there are 7 cities": you just look at where the lights are crowded.

Examples of density-based algorithms: **DBSCAN** and **OPTICS**.

The full name is **D**ensity-**B**ased **S**patial **C**lustering of **A**pplications with **N**oise. Once you finish this chapter every word will make sense:

Word	Meaning here
Density-Based	clusters are crowded regions
Spatial	works on points in space (distance between points)
Clustering	grouping points
Applications	real datasets (geo-data, customers, images...)
with Noise	it explicitly marks points that belong to no cluster

2.3 The two knobs: ε (epsilon) and MinPts

Your goal: decide for any point whether it is in a dense or a sparse area. Two **hyperparameters** let us do that:

- **ε (Epsilon, eps):** a radius. The unit does not matter (1 cm, 1 mm, 1 km: it is whatever unit your data uses).
- **MinPts (min_samples):** a minimum count of points. If a circle holds MinPts points or more, the place is *dense*; otherwise *sparse*.

$$N_\varepsilon(p) = \{\, q \in D \,:\, \text{dist}(p, q) \le \varepsilon \,\}$$

ε-neighbourhood

D = the dataset, p = a point, $N_\varepsilon(p)$ = all points within distance ε of p (the points inside the circle of radius ε around p, **including p itself**). The "density" around p is simply $|N_\varepsilon(p)|$, the number of points inside.

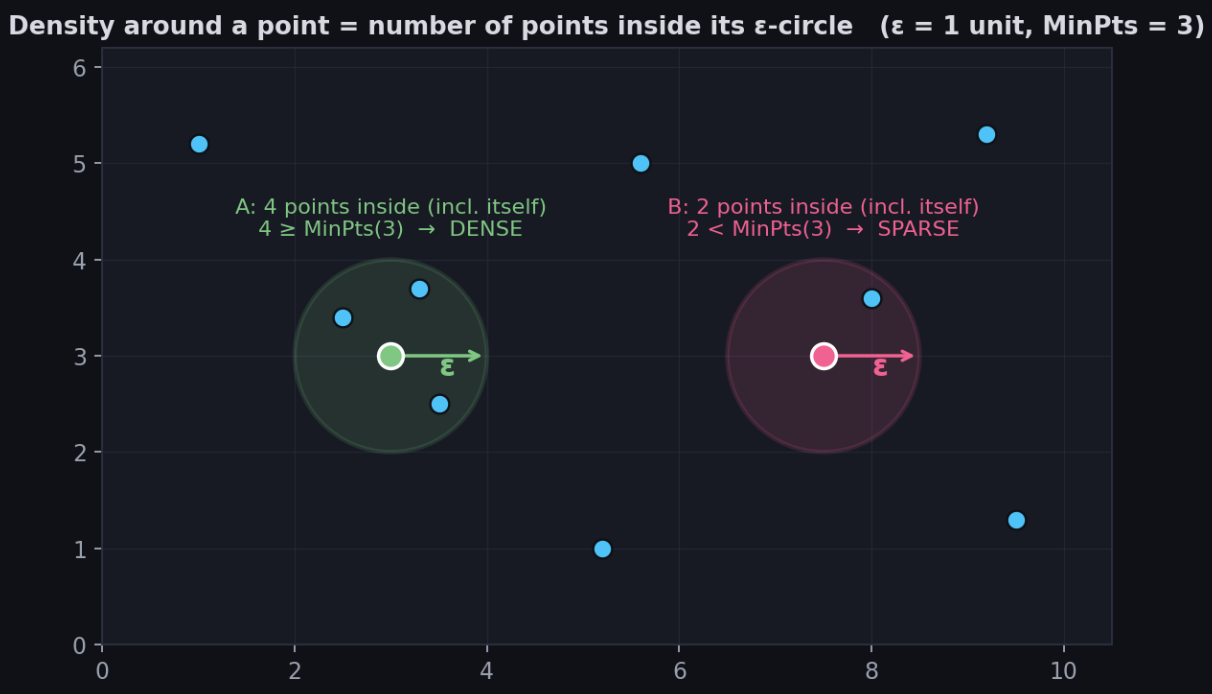


Figure 2.2 – With ε = 1 and MinPts = 3: the circle around A holds 4 points (dense); the circle around B holds only 2 (sparse).

Example from the lecture: assume ε = 1 unit and MinPts = 3. To judge a point, draw a circle of radius ε around it and count. The first circle contains **4** points, which is greater than 3, so the density around this point is **dense**. A second point's circle holds only **2** points, less than 3, so it is **sparse**. In this way you can calculate the density around any point.

> **NOTE**
> **Counting convention.** scikit-learn counts the point itself inside min_samples. We do the same everywhere in this tutorial, so "5 points in the circle" includes the centre point.

> **REMEMBER**
> ε and MinPts are your **two hyperparameters**. There are only two to tune, but that is exactly why learning both properly matters.

2.4 Three kinds of points: core, border, noise

Every point in DBSCAN gets exactly one of three roles. The concept is simple if you know ε and MinPts.

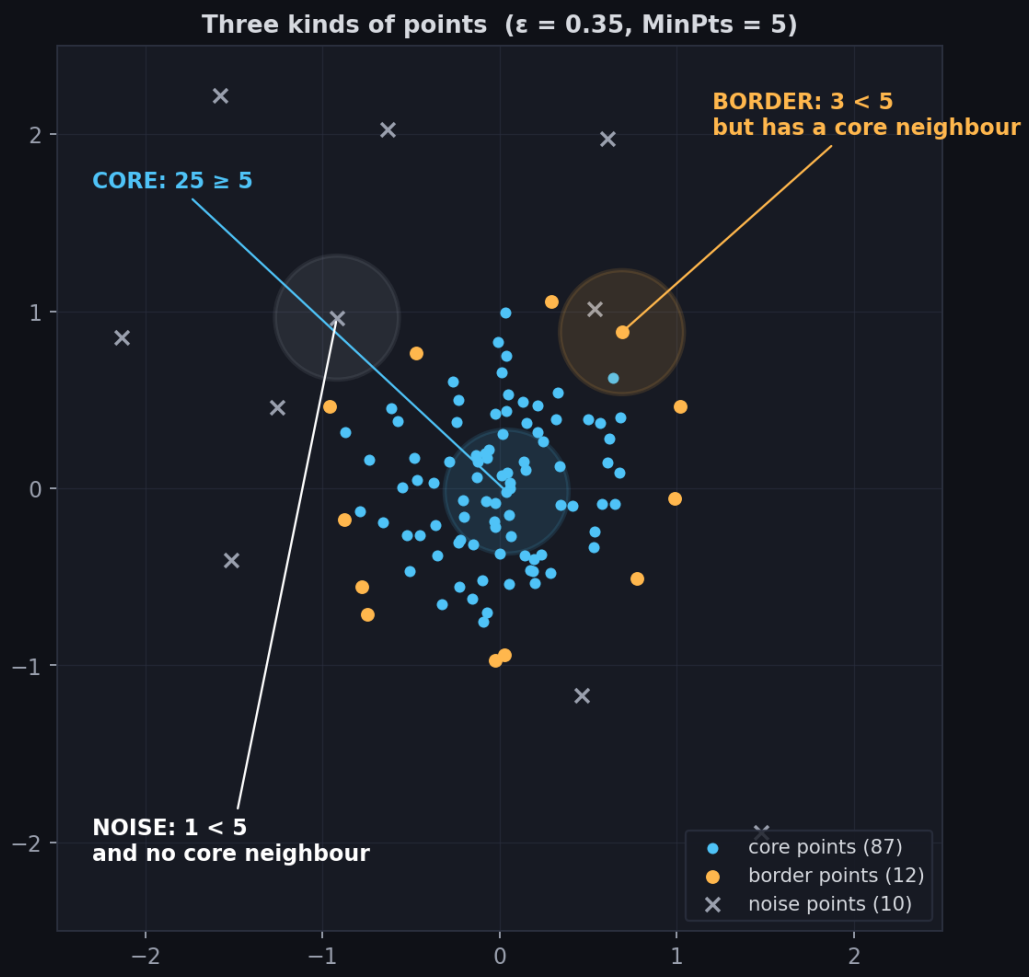


Figure 2.3 – ε = 0.35, MinPts = 5. The shaded circles are ε-neighbourhoods of one example of each kind. Counts include the point itself.

Core point

$$p \text{ is a core point} \iff |N_\varepsilon(p)| \;\geq\; \text{MinPts} \qquad \text{core}$$

A core point has at least MinPts points (itself included) in its ε-neighbourhood. Lecture example: MinPts = 5. Draw the ε circle around a point; if you count 1, 2, 3, 4, 5, then since 5 is equal to MinPts (not more, but equal) the point is a **core point**.

Border point

Two conditions, both required: **(1)** it has *fewer* than MinPts points in its ε-neighbourhood, **(2)** but at least one **core point** lies inside that neighbourhood. Lecture example: the circle holds 3 points (< 5), so it is not core, but one of those points is itself a core point. Both criteria hold, so it is a **border point**.

$$p \text{ is a border point} \iff |N_\varepsilon(p)| < \text{MinPts} \;\text{ and }\; \exists\, q \in N_\varepsilon(p) :\; q \text{ is core} \qquad \text{border}$$

Noise point

Neither core nor border. Lecture example: the circle holds only 2 points (< 5), so it cannot be core; and no core point sits in its neighbourhood, so it is not border either. What is left? **Noise**.

Type	Points in ε-circle	Has a core neighbour?	Role in the cluster
Core	≥ MinPts	(not needed)	the **interior**; clusters grow through core points
Border	< MinPts	yes, at least one	the **edge**; joins a cluster but cannot grow it
Noise	< MinPts	no	belongs to no cluster; label −1

On a real dataset: the points forming the body of the cluster are core (blue in Figure 2.3), the points on its periphery are border (orange), and the far-flung outliers are noise (grey crosses). Tomorrow, whenever you cluster a dataset, you will see this categorisation.

2.5 Density-connected points

One last concept. If two points A and B are **density-connected**, it simply means you can pick both of them up and put them in **one cluster**. How do we know? If A and B are linked *indirectly* through a series of **core points**, and every adjacent pair of those core points is at a distance of at most ε, then A and B are density-connected.

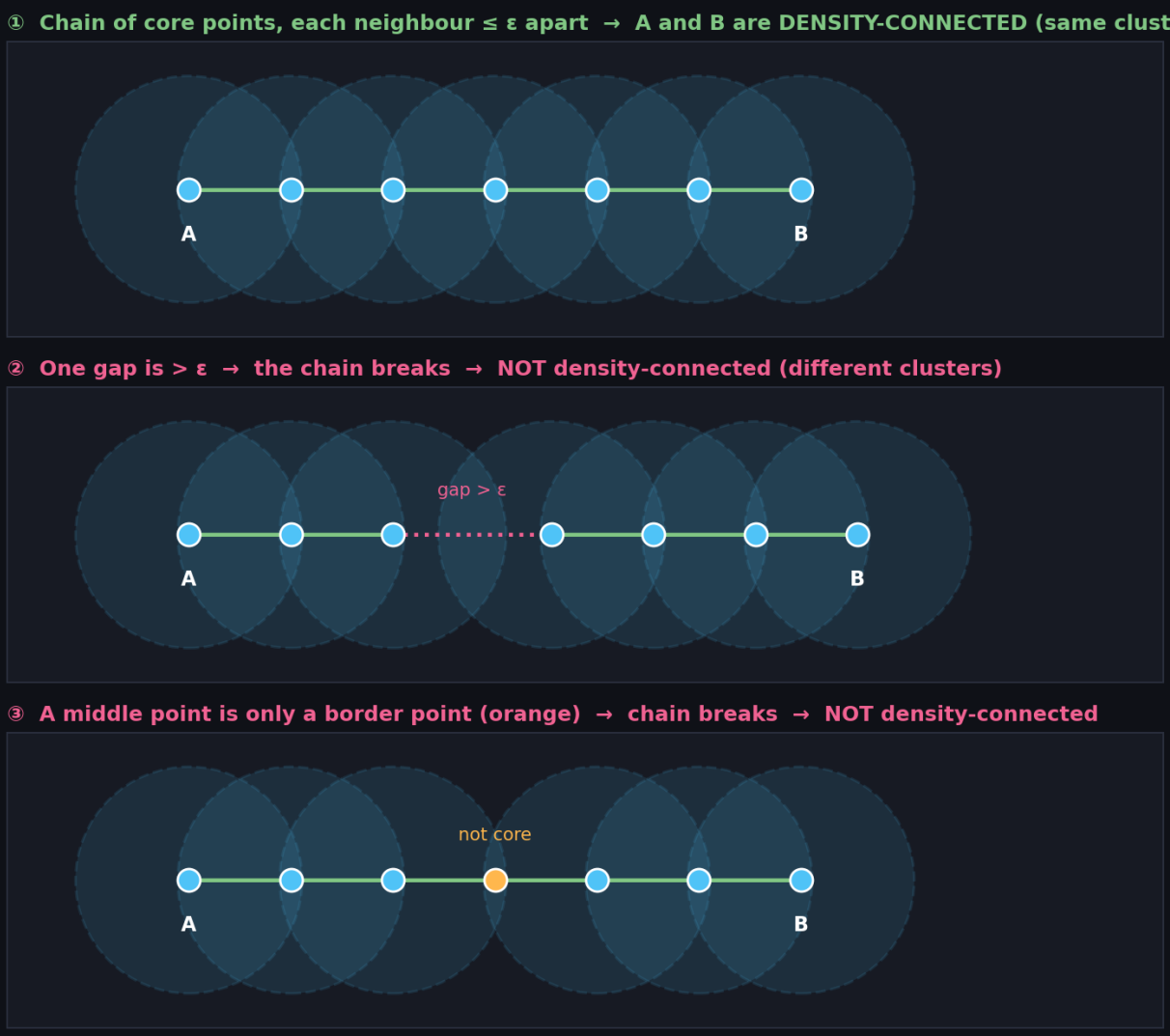


Figure 2.4 – ① a chain of core points ≤ ε apart connects A and B. ② A gap larger than ε breaks it. ③ A non-core (border) point in the middle also breaks it.

The path **breaks** in exactly two situations:

1. a point in the middle is **not a core point** (it is a border or noise point), or
2. two core points in the chain are **more than ε apart**.

Then A and B are not density-connected, and you will have to put them in different clusters.

> **MATH**
> **Formal definitions (from the DBSCAN paper).**
> • *q is directly density-reachable from p* if $q \in N_\varepsilon(p)$ and p is a core point.
> • *q is density-reachable from p* if there is a chain $p = p_1, p_2, \ldots, p_m = q$ where each p_{i+1} is directly density-reachable from p_i.
> • *p and q are density-connected* if some point o exists from which both p and q are density-reachable.
> **A DBSCAN cluster** = a maximal set of mutually density-connected points. (The lecture's "chain of core points" is the same thing, in plain words.)

We now know MinPts, ε, the three point types and density-connectivity. We are ready for the algorithm.

2.6 The DBSCAN algorithm (4 steps)

The whole algorithm runs in four steps:

1. **Label every point** as core, border or noise using ε and MinPts (count the points in each ε-circle).
2. **Pick an unvisited core point** and start a **new cluster** with it (give it a new cluster id).
3. **Expand the cluster:** add all points inside its ε-circle; for every *core* point added, add its neighbours too; keep going until no more points can be added. (Border points are added but do not expand further.)
4. **Repeat** from step 2 with the next unvisited core point, until all core points are visited. Points never reached by any cluster are **noise (−1)**.

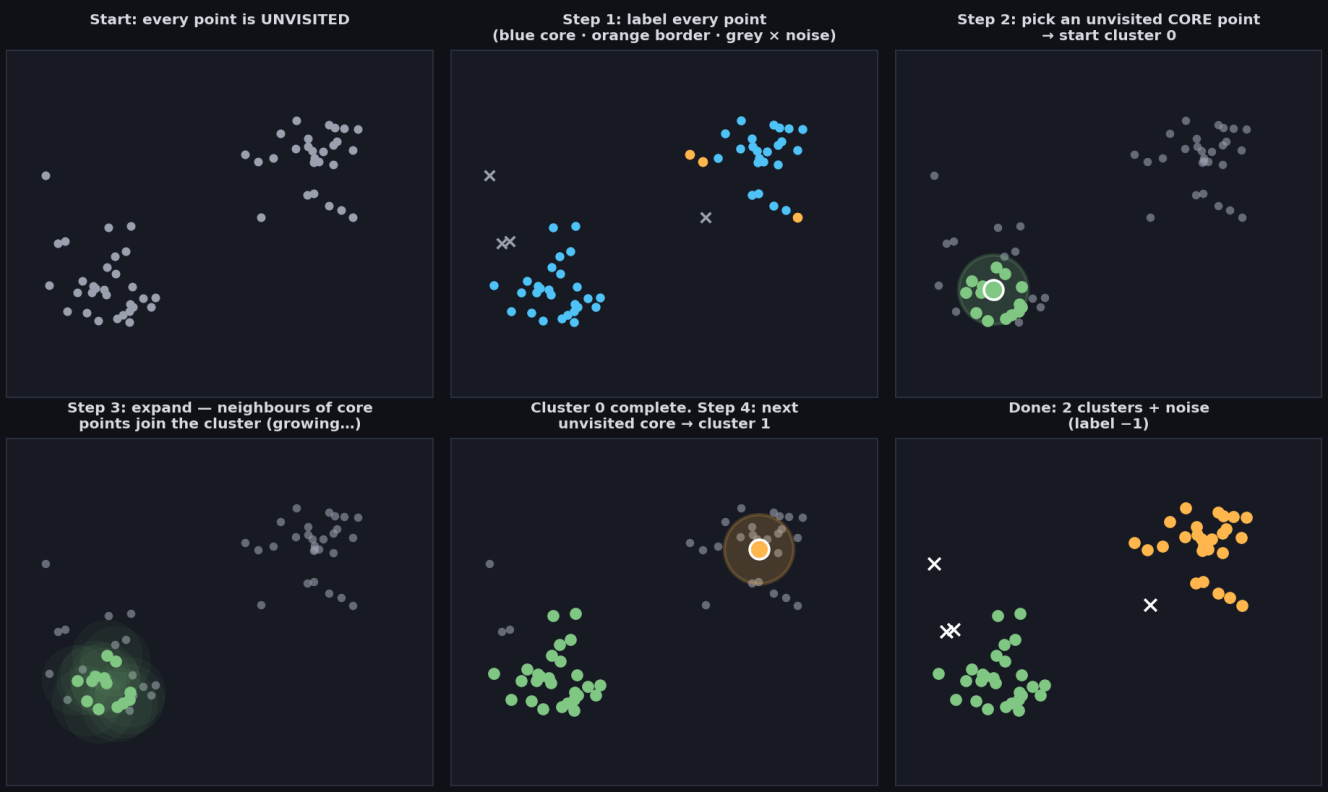


Figure 2.5 – DBSCAN on a toy dataset (ε = 0.7, MinPts = 4): label → seed → expand → next cluster → done.

pseudocode

```
for each point p in D:
    if p already visited or p is not core:  continue
    start new cluster C with p
    queue = [p]
    while queue not empty:
        q = queue.pop()
```

```
        if q is core:                       # only core points spread the cluster
            for r in N_eps(q):
                if r has no cluster yet:
                    put r in C ; queue.append(r)
# every point still without a cluster is noise  (label -1)
```

> **WATCH OUT**
> A border point sitting between two clusters is given to whichever cluster reaches it *first*, so the result can depend on the order of the data. Core points and noise are order-independent.

Complexity. The expensive part is finding each point's neighbours. With a spatial index (KD-tree / ball-tree) the typical cost is about O(n log n); without one, brute force is $O(n^2)$ (and in very high dimensions indexes stop helping).

2.7 DBSCAN in scikit-learn

Very simple, as scikit-learn makes everything. The example is from scikit-learn's own documentation: **six points**, three of them close together at the lower left, two close together in the middle, and one far away (noise).

dbscan_basic.py

```
import numpy as np
from sklearn.cluster import DBSCAN

X = np.array([[1, 2], [2, 2], [2, 3],
              [8, 7], [8, 8], [25, 80]])

db = DBSCAN(eps=3, min_samples=2).fit(X)   # the two hyperparameters
print(db.labels_)                          # the whole clustering result lives here
```

```
[ 0  0  0  1  1 -1]
```

The first three points form cluster 0, the next two form cluster 1, and the last is labelled **−1**: DBSCAN gives every noise point the label −1. (Note: the trained model has `labels_`; there is no `predict` method. See limitations.)

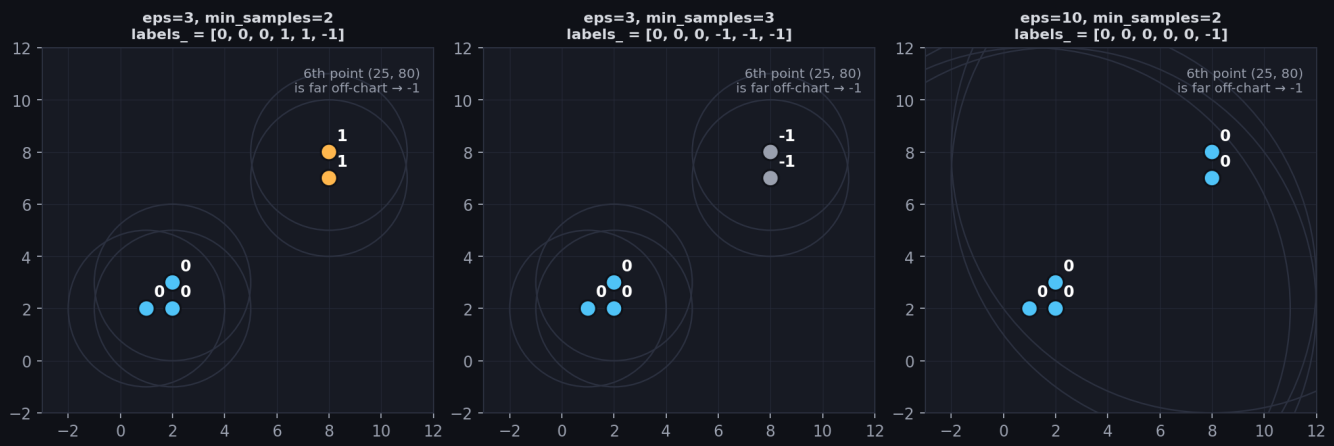


Figure 2.6 – The same six points with three settings. Circles are ε-neighbourhoods; the number next to each point is its label.

Quick question from the lecture: what if min_samples = 3 instead of 2? Pause and think, then check:

min_samples = 3

```
DBSCAN(eps=3, min_samples=3).fit(X).labels_      # -> [ 0  0  0 -1 -1 -1]
```

Only the three lower-left points stay in a cluster. The two middle points are now **noise**: within their ε-neighbourhood there can never be 3 points (there are only 2 of them there), so they are not core, and

they cannot be border either because no core point is nearby.

And if we increase ε to 10 (bigger circles)?

```
eps = 10
```

```
DBSCAN(eps=10, min_samples=2).fit(X).labels_     # -> [ 0  0  0  0  0 -1]
```

The neighbourhood becomes so big that the first three points reach the middle two, and all five points collapse into **one** cluster (the scale of the plot is 0 to 10 on the clustered part). The far point (25, 80) is still noise. Try changing ε and min_samples yourself, since they are hyperparameters and you have the control.

A proper example: concentric circles

dbscan_circles.py

```
from sklearn.datasets import make_circles
X, _ = make_circles(n_samples=400, factor=0.4, noise=0.05, random_state=1)
labels = DBSCAN(eps=0.2, min_samples=5).fit_predict(X)     # two clusters, one per ring
```

This is the first dataset where K-Means failed (Figure 0.2). DBSCAN separates the rings perfectly, and the specialty is that it **does not care about the shape or size** of the data, only about density. (The bottom-left panel of Figure 0.2 is exactly this run.) Explore with other datasets: that is how you learn.

Useful attributes and a from-scratch version

- `labels_`: cluster id per point, −1 = noise
- `core_sample_indices_`: which points are core
- `components_`: the core points' coordinates

dbscan_scratch.py

```
import numpy as np
from collections import deque

def my_dbscan(X, eps, min_pts):
    n = len(X)
    D = np.linalg.norm(X[:, None] - X[None], axis=2)
    neighbours = [np.where(D[i] <= eps)[0] for i in range(n)]      # eps-neighbourhoods
    is_core = np.array([len(nb) >= min_pts for nb in neighbours])   # Step 1: core points
    labels = np.full(n, -1)                                          # -1 = noise (for now)
    cluster = 0
    for i in range(n):
        if labels[i] != -1 or not is_core[i]:
            continue                                                 # only unvisited core points start clusters
        labels[i] = cluster                                          # Step 2: new cluster
        queue = deque([i])
        while queue:                                                 # Step 3: expand
            j = queue.popleft()
            if not is_core[j]:
                continue                                             # border points join but do not expand
            for k in neighbours[j]:
                if labels[k] == -1:
                    labels[k] = cluster
                    queue.append(k)
        cluster += 1                                                 # Step 4: repeat
    return labels
```

Output on the six-point example: `[ 0 0 0 1 1 -1]`, identical to scikit-learn (also verified on a 300-point moons dataset: 100% identical labels).

2.8 Choosing ε and MinPts (Level 3 · Pro)

DBSCAN is very sensitive to its hyperparameters. This figure changes only ε on the same data:



Figure 2.7 – Same moons data, MinPts = 5. Tiny ε → almost everything is noise or shattered; just right → 2 moons; too big → merged into one.

MinPts rule of thumb

- Minimum is 2 or 3 in toy data; a common default is **MinPts ≥ D + 1** where D = number of dimensions, and often **2 × D** for noisy data.
- More data or more noise → larger MinPts. Larger MinPts makes the algorithm more conservative (more noise, fewer small clusters).

ε from the k-distance graph

For every point compute the distance to its *k-th nearest neighbour* (k = MinPts), sort those distances from largest to smallest and plot them. Points inside clusters have small values; noise has large values. The **knee (elbow)** of the curve is a good ε:

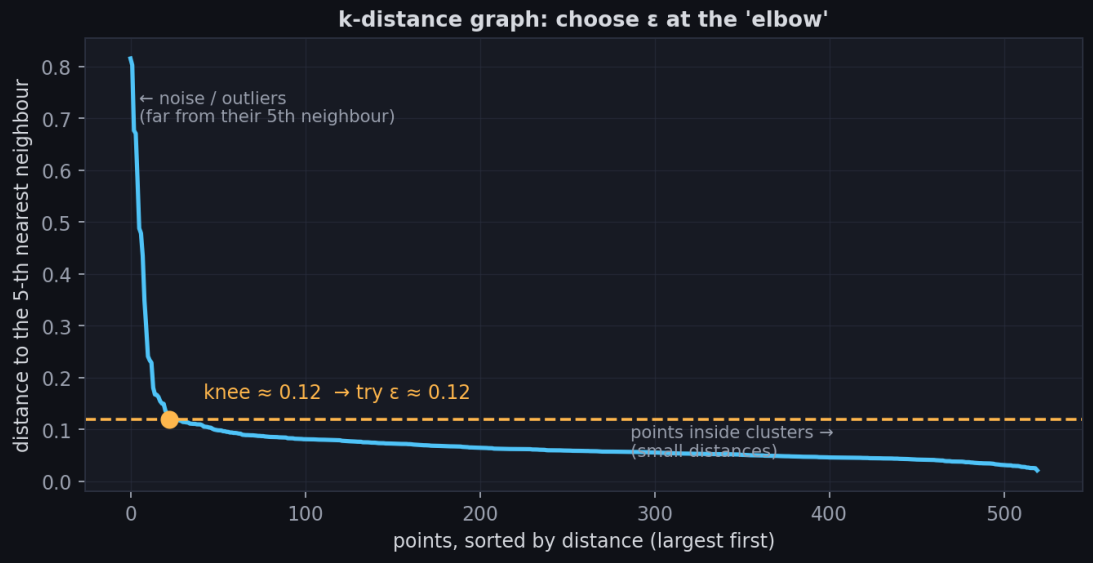


Figure 2.8 – The k-distance graph. The sharp bend separates cluster points (flat part) from noise (steep part).

k_distance.py

```
from sklearn.neighbors import NearestNeighbors
k = 5
dist, _ = NearestNeighbors(n_neighbors=k).fit(X).kneighbors(X)
kdist = np.sort(dist[:, -1])[::-1]       # distance to the k-th neighbour, largest first
plt.plot(kdist); plt.ylabel(f"{k}-th NN distance"); plt.show()   # eps = height of the knee
```

WATCH OUT

Scale first! ε is one number applied to all features, so features must be on comparable scales (use StandardScaler) or one feature will decide everything.

2.9 Advantages and limitations

Advantages

1. **Robust to outliers.** It has a built-in ability to identify outliers, separately detecting noise points. That is why it is also used in *anomaly detection* scenarios, where K-Means was not good.
2. **No need to tell the number of clusters.** It works it out itself, a big headache removed (in the 6-point example it figured out two clusters on its own).
3. **Arbitrarily shaped clusters.** Give it any type of data and it clusters: shape does not matter, because the idea is based on density.
4. **Only two hyperparameters** (MinPts and ε) to learn, unlike algorithms with many knobs.

Limitations

1. Very sensitive to hyperparameters. A small change in ε or MinPts can produce a very different clustering (Figure 2.7). Be super careful when choosing both.

2. Clusters of different density. If one cluster is sparse (open) and another dense (tightly packed), a **single** ε cannot suit both.

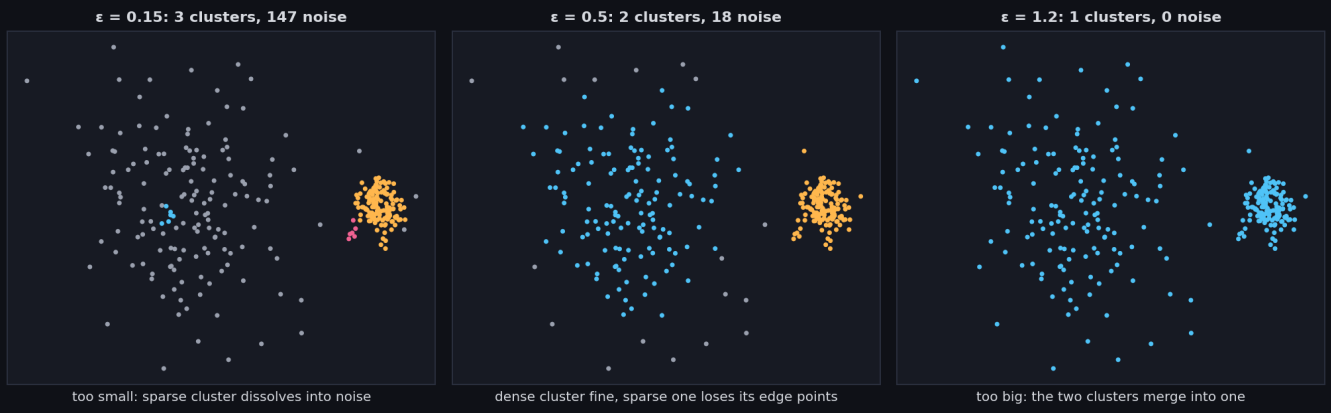


Figure 2.9 – A sparse blob and a dense blob. Small ε: the sparse blob falls apart into noise. Large ε: the two blobs merge. No single ε is right for both.

3. It cannot make predictions. There is no `predict` function: it only runs on the training data and clusters it. If a new point arrives tomorrow it cannot tell you where it goes: you would have to redo the calculations (re-cluster / re-train). So it is useful for clustering, not for prediction.

> **PRO TIP**
> **Workarounds.** (a) Give the new point the label of its nearest *core* sample if it is within ε, else noise. (b) Train a classifier (k-NN) on DBSCAN's labels. (c) Use **HDBSCAN**, which supports approximate prediction.

Other pro-level notes. In very high dimensions distances become similar (curse of dimensionality), so density is hard to define: reduce dimensions first (PCA/UMAP). For latitude/longitude data use `metric='haversine'` (in radians). To handle varying densities use **OPTICS** (orders points by reachability distance, so one run covers many ε) or **HDBSCAN** (builds a hierarchy over DBSCAN at all ε, keeps the most stable clusters: it only needs min_cluster_size).

2.10 A visualisation tool that makes it click

There is a website where you can see DBSCAN working through animation (the well-known “Visualizing DBSCAN Clustering” page by Naftali Harris: search for that title). Many datasets are provided, for example **Smile Face**, and you set ε and MinPts with sliders. Press **Go** and you actually see the algorithm run: the circles move forward, add points to the cluster, stop, and a new cluster starts. About 9–10 datasets are available.

The best way to use it (the lecturer tested all datasets and wrote observations in a notebook, which helped a lot):

Dataset	Experiment	What you should notice
Smile Face	Start ε small, increase slowly	face parts separate, then merge into one blob at big ε
Any dataset	Keep ε fixed, raise MinPts	more noise points appear; small clusters vanish
Uniform / random	Try any ε	very hard to get meaningful clusters when there is no density structure
Different-density data	Try to satisfy both clusters	you can’t: Figure 2.9 live

Chapter 2 summary

- DBSCAN = density-based. Two knobs: ε (radius) and MinPts (count).
- Core (≥ MinPts in circle), border (fewer, but next to a core), noise (neither).
- Cluster = chain of density-connected core points (+ their border points). Algorithm: label → seed → expand → repeat.
- Pros: finds noise, any shape, no k. Cons: sensitive to ε/MinPts, struggles with varying density, no predict.

Chapter 3 · Putting It All Together

3.1 K-Means vs Hierarchical vs DBSCAN

Criterion	K-Means	Hierarchical (agglomerative)	DBSCAN
Needs k up front?	Yes	No (cut the dendrogram; or give n_clusters)	No
Main idea	distance to centroids	merge closest clusters, tree	crowded regions
Cluster shapes	round / convex only	depends on linkage (single: arbitrary)	arbitrary
Outliers	very sensitive	single: sensitive; complete/Ward: ok	labelled as noise
Hyperparameters	k	linkage, metric, k or distance_threshold	ε, MinPts
Different densities	ok if round	ok	weak (HDBSCAN fixes)
Scales to big n?	Yes (fast)	No: $O(n^2)$ memory	Medium: O(n log n) with an index
Predicts new points?	Yes	No (no predict)	No (no predict)
Bonus output	centroids	full hierarchy / dendrogram	noise / anomaly flags

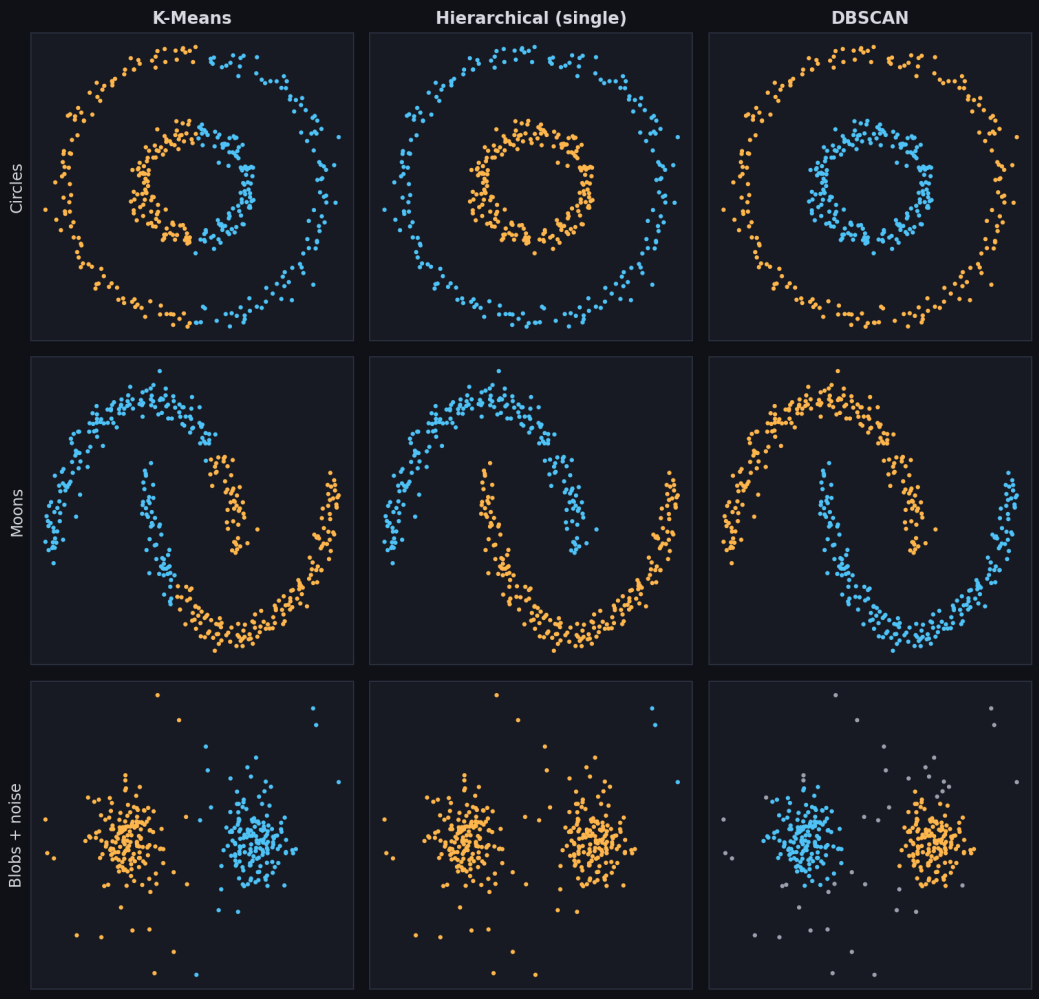


Figure 3.1 – Same three datasets, three algorithms. Single-link hierarchical and DBSCAN follow shapes; K-Means does not; only DBSCAN rejects noise.

3.2 Which one should I use?

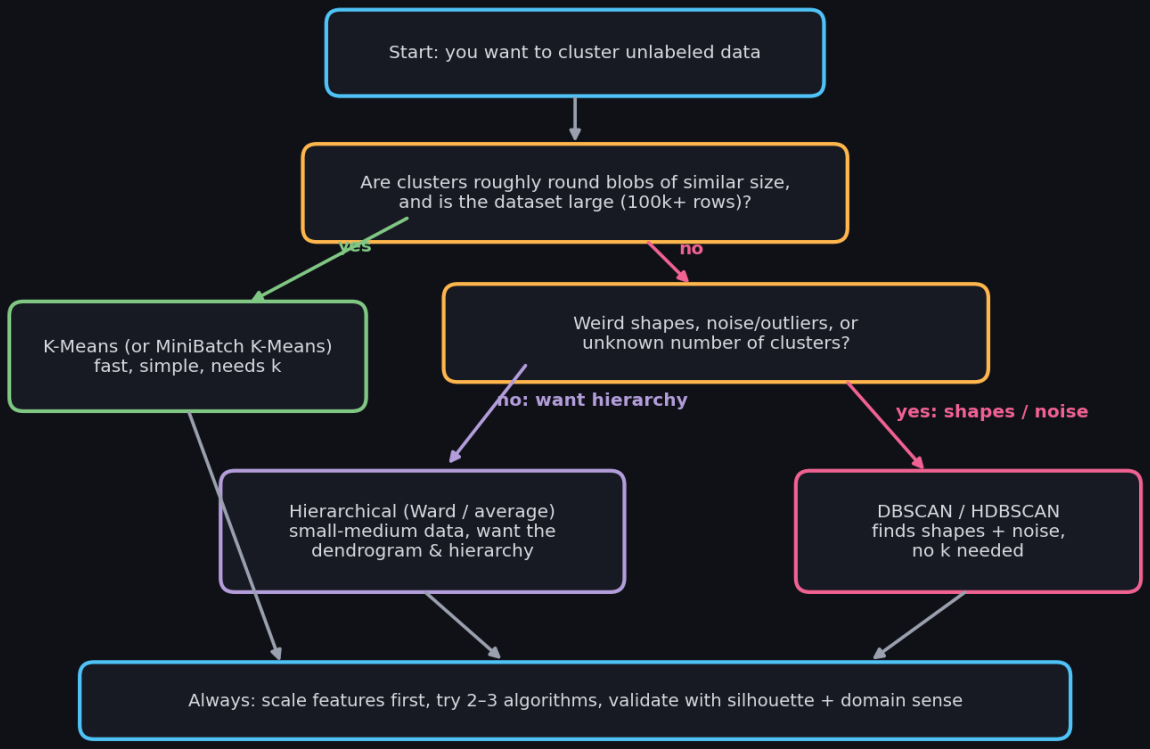


Figure 3.2 – A practical decision guide.

3.3 Interview questions (with short answers)

Q. Why do we need hierarchical clustering when we have K-Means?

K-Means needs k, assumes round clusters, is sensitive to outliers. Hierarchical needs no k up front, can handle other shapes via linkage choice, and gives a dendrogram.

Q. Agglomerative vs divisive?

Agglomerative merges bottom-up (n clusters → 1). Divisive splits top-down (1 → n). Agglomerative is more common because merging by distance is easier than choosing splits ($2^{n-1}-1$ possibilities).

Q. What is a dendrogram, and how do you pick k from it?

A tree of merges with merge distance on the y-axis. Cut through the longest vertical line not crossed by any horizontal line; count the vertical lines cut.

Q. Explain single, complete, average and Ward.

Cluster distance = min pair / max pair / mean of all pairs / increase in within-cluster variance. Single: chaining and noise-sensitive. Complete: robust to noise but breaks big clusters. Average: in between. Ward: default, compact equal clusters.

Q. Why can’t hierarchical clustering handle millions of points?

It stores an n×n distance matrix: $O(n^2)$ memory (and $\geq O(n^2)$ time).

Q. What are ε and MinPts?

ε = neighbourhood radius; MinPts = minimum points inside it for the area to count as dense.

Q. Core vs border vs noise?

Core: ≥ MinPts in ε-circle. Border: fewer, but a core point is inside its circle. Noise: neither.

Q. What does density-connected mean?
Two points linked by a chain of core points whose consecutive distances are ≤ ε; they end up in the same cluster.

Q. Does DBSCAN need k? How does it treat outliers?
No k. Outliers become noise (label −1), so it is also used for anomaly detection.

Q. Weaknesses of DBSCAN?
Sensitive to ε/MinPts, struggles with clusters of different density, high dimensions, and has no predict method.

Q. How would you choose ε?
k-distance graph (k = MinPts) and take the knee; validate with domain sense/silhouette on non-noise points.

Q. When would DBSCAN fail but hierarchical (Ward) win, and vice versa?
Ward wins on equal-size, touching round blobs with different densities. DBSCAN wins on arbitrary shapes with noise.

3.4 Cheat sheet

Topic	Remember
Agglomerative	start n clusters → merge closest pair → … → 1 cluster; n−1 merges; record in dendrogram
Proximity matrix	n × n distances, diagonal 0, update merged row/col each step
Linkage formulas	single: min d(a,b) · complete: max d(a,b) · average: mean d(a,b) · Ward: $\lvert A\rvert\lvert B\rvert/(\lvert A\rvert+\lvert B\rvert)\cdot\lVert\mu_A-\mu_B\rVert^2$
Choose k	longest free vertical line; or distance_threshold; or silhouette
sklearn	AgglomerativeClustering(n_clusters, metric/affinity, linkage, distance_threshold)
Limit (hier.)	$O(n^2)$ memory → small/medium data only
DBSCAN	ε-neighbourhood $N_\varepsilon(p)$; core if $\lvert N_\varepsilon(p)\rvert \geq$ MinPts; border: next to core; noise: −1
DBSCAN sklearn	DBSCAN(eps, min_samples).fit(X).labels_ ; no predict
Tune ε	k-distance knee; scale the features

3.5 Practice and mini-projects

1. Redo the 5-point worked example with **complete link** and verify the heights 0.707, 1.118, 2.236, 6.519 from the table in 1.7.
2. Run Ward on the Mall data with k = 3, 4, 5, 6 and compare mean silhouette scores. Does the dendrogram rule agree?
3. **Lecture assignment:** cluster countries (GDP, population, area) and find which countries share a cluster with India; read the dendrogram for India's nearest neighbours.
4. Generate `make_moons` and `make_circles`; find ε values where DBSCAN works and where it breaks. Plot the k-distance graph to predict a good ε.
5. On the Mall data (scaled), run DBSCAN and see which customers are labelled noise. Do they look like anomalies?
6. Open the DBSCAN visualiser and write observations for every dataset (the lecturer's own study method).

7. Extend the from-scratch agglomerative code to support Ward and compare with SciPy.

3.6 Glossary and notation

Term / symbol	Meaning
Cluster	a group of similar points
Centroid μ	mean of a cluster's points
Proximity matrix	$n \times n$ table of pairwise distances
Linkage	rule for the distance between two clusters
Dendrogram	tree recording merges and their distances
Inter-cluster distance	the vertical axis of the dendrogram
SSE	sum of squared distances to the cluster mean
ε (eps)	neighbourhood radius in DBSCAN
MinPts (min_samples)	points needed in an ε-circle to be dense
$N_\varepsilon(p)$	set of points within ε of p
Core / border / noise	DBSCAN point roles
Density-connected	joined by a chain of core points $\leq \varepsilon$ apart
Noise label	-1 in scikit-learn

CORRECTION / UPDATE

Summary of the clarifications in this document: (1) memory for 10^6 points is ~8 TB, not ~10 GB; (2) scikit-learn's `affinity` is now `metric`; (3) Ward is stated with its exact formula; (4) the spoken script announced "four steps" of DBSCAN without listing them, so they are written out in 2.6; (5) the Mall-customer figures are a synthetic stand-in for the real CSV.

You have now climbed the whole ladder: from "what is clustering" to merge trees, linkage maths, density, and tuning. Keep experimenting.